

Tytuł pracy: Oprogramowanie służące do komunikacji i zarządzania ocenami w środowisku edukacyjnym

Streszczenie:

Niniejsza praca omawia zagadnienia związane z narzędziem służącym do informowania, zarządzania oraz przekazywania ocen w środowisku edukacyjnym. Dokument prowadzi czytelnika zgodnie z zasadą „od ogółu do szczegółu”, obejmując takie aspekty jak geneza i cel powstania narzędzia, a także techniczne rozwiązania zastosowane przy jego tworzeniu.

Słowa kluczowe:

Narzędzie dla nauczycieli, automatyzacja procesów edukacyjnych, interakcja nauczyciel-uczeń, zarządzanie ocenami, aplikacja webowa

Thesis title:

Software for communication and assessment management in an educational environment

Abstract:

This paper discusses issues related to a tool for informing, managing and communicating assessments in educational settings. The paper guides the reader along the principle of "from the general to the specific", covering such aspects as the genesis and purpose of the tool, as well as the technical solutions used in its development.

Keywords:

Tool for teachers, Automation of educational processes, Teacher-student interaction, Assessment management, Web application

Projekt inżynierski:

Oprogramowanie służące do komunikacji i zarządzania ocenami w środowisku edukacyjnym

kierujący projektem: dr inż. Mariusz Pleszczyński

Kamil Żebrok

Opis wkładu: interfejs użytkownika, integracja komponentów

Piotr Glajcar

Opis wkładu: backend, integracja z USOS API, wdrożenie

Spis treści

1. Wstęp	9
1.1. Geneza pracy	9
1.2. Cel i założenia pracy	9
1.3. Zakres pracy, architektura systemu, technologie	10
1.4. Przegląd istniejących rozwiązań	10
2. Oprogramowanie	13
2.1. Wymagania	13
2.1.1. Środowisko deweloperskie	13
2.2. Korzystanie z aplikacji	14
2.2.1. Start i logowanie	14
2.2.2. Panel prowadzących	15
2.2.3. Panel uczestników	18
3. Interfejs Użytkownika	21
3.1. Narzędzia	21
3.2. Style i kolorystyka	22
3.3. Struktura plików związanych z interfejsem	22
3.3.1. App.jsx	24
3.3.2. ProtectedRoute.jsx	24
3.3.3. Logowanie, ścieżki publiczne i pomocnicze	25
3.3.4. Strony główne użytkowników	27
3.3.5. Moduł prowadzących – dostępne ścieżki i funkcjonalności	28
3.3.6. Moduł uczestników – system przeglądania i rejestracji	32
4. Backend	35
4.1. Rola backendu w aplikacji	35
4.2. Python Virtual Environment	35
4.3. Architektura backendu – Django i Django Rest Framework	36
4.3.1. Struktura projektu Django	36
4.4. Omówienie bazy danych	37
4.4.1. Struktura bazy danych	38
4.4.2. Struktura encji użytkownika	38
4.4.3. Relacje między encjami	38
4.4.4. Relacja między Użytkownik (User), Student i Nauczyciel (Teacher) (1:1)	40
4.4.5. Relacja między Nauczyciel (Teacher) i Kurs (Course) (1:M)	41

4.4.6. Relacja między Student (Student) i Kurs (Course) poprzez Rejestrację (Enrollment) (M:N)	41
4.4.7. Relacja między Zadanie (Assignment) i Kurs (Course) (1:M) . .	41
4.4.8. Relacja między Rejestracja (Enrollment), Zadanie (Assignment) i Ocena (Grade) (1:M)	41
4.4.9. Podsumowanie relacji między encjami	41
4.5. Modele	42
4.5.1. Modele Django a encje bazy danych	42
4.5.2. Przykładowa implementacja modelu	43
4.5.3. Identyfikator główny w modelach	43
4.5.4. Relacja z modelem użytkownika	44
4.5.5. Znaczenie <code>on_delete=models.CASCADE</code>	44
4.5.6. Modele wykorzystujące <code>on_delete=models.CASCADE</code>	44
4.5.7. Metoda <code>__str__</code>	45
4.6. Django Admin	45
4.6.1. Rejestracja modeli w Django Admin	46
4.6.2. Konfiguracja widoków w Django Admin	46
4.7. OAuth 1.0a – protokół autoryzacji	50
4.7.1. Uzyskanie tymczasowego tokenu żądania	50
4.7.2. Autoryzacja	51
4.7.3. Wymiana tokenów	52
4.7.4. Wykorzystanie tokenu dostępu	52
4.8. Integracja aplikacji z USOS API	53
4.8.1. Uzyskanie klucza konsumenta i consumer secrets	53
4.8.2. Dane użytkownika pobierane z USOS API	54
4.8.3. Proces inicjalizacji OAuth	55
4.8.4. Proces autoryzacji w funkcji <code>initiate_oauth</code>	56
4.8.5. Pobieranie tokena dostępu	57
4.8.6. Pobieranie informacji o użytkowniku	58
4.8.7. Przetwarzanie danych użytkownika	58
4.8.8. Określanie roli użytkownika	59
4.8.9. Tworzenie obiektów studenta i nauczyciela	60
4.8.10. Logowanie użytkownika i przekierowanie	60
4.9. Pliki w projekcie backendu	60
4.9.1. <code>manage.py</code>	61
4.9.2. <code>serializers.py</code>	61
4.9.3. <code>responses.py</code>	62

4.9.4. views.py	63
4.9.5. urls.py	64
4.9.6. settings.py	64
5. Wdrożenie aplikacji	67
5.1. Wybór bazy danych: PostgreSQL zamiast SQLite	67
5.2. Zmienne środowiskowe – bezpieczeństwo i elastyczność	68
5.2.1. Struktura plików środowiskowych w aplikacji	68
5.2.2. Plik środowiskowy dla frontendu	69
5.2.3. Plik środowiskowy dla backendu	69
5.2.4. Omówienie wybranych zmiennych	70
5.2.5. Pobieranie zmiennych środowiskowych w Django	70
5.2.6. Korzyści wynikające z zastosowania plików <code>.env</code>	71
5.3. Budowanie frontendu i integracja z backendem	71
5.4. Konfiguracja ciągłego wdrożenia (Continuous Deployment)	72
5.5. Plik pre-wdrożeniowy <code>build.sh</code>	72
5.6. Uruchamianie aplikacji na Render.com	73
5.6.1. Plik ASGI w Django	74
5.6.2. Testowanie wdrożenia na Render.com	74
5.7. SOP i CORS	74
5.7.1. CORS – kontrolowane udostępnianie zasobów	75
5.7.2. Alternatywne rozwiązanie i problem XSS	75
5.7.3. Praktyki zapewniające bezpieczeństwo	76
6. Narzędzia wykorzystane w procesie rozwoju aplikacji	77
6.1. Git	77
6.2. Kanban w Taiga.io	77
6.3. Web Inspector	78
6.4. Postman	79
6.5. Debugger VSCode	79
7. Podsumowanie	81
Literatura	83

1. Wstęp

Praca dotyczy opracowanego narzędzia usprawniającego procesy w środowisku edukacyjnym, które ułatwia zarówno sprawne przekazywanie informacji przez prowadzących kursy, jak i ich odbiór przez kursantów. Oprogramowanie umożliwia edukatorom zarządzanie kursami, wprowadzanie kluczowych elementów, takich jak etapy, efekty, punkty i sprawdziany, a kursantom śledzenie ich postępów i wyników, co zapewnia przejrzystość procesu nauczania. Prowadzący powinien zapoznać uczestników z działaniem oprogramowania na zajęciach organizacyjnych, aby zapewnić im dostęp do własnych kont, na których dokumentowane będą ich indywidualne postępy.

1.1. Geneza pracy

Wraz z dynamicznym rozwojem technologii wzrasta jakość nauczania i łatwość komunikacji w środowiskach edukacyjnych. Jednak coraz większa złożoność narzędzi cyfrowych może być zniechęcająca zarówno dla studentów, jak i prowadzących zajęcia. Często pojawiają się skargi na zbyt skomplikowane oprogramowanie, które wymaga dużo czasu i uwagi, odciągając od głównego celu – edukacji. Ten problem zainspirował powstanie tej pracy, której celem jest stworzenie prostego, intuicyjnego narzędzia, wspierającego proces nauczania.

1.2. Cel i założenia pracy

Celem pracy jest w pełni funkcjonalne oprogramowanie, wspierające codzienną pracę dydaktyczną. Program umożliwia prowadzącym szybkie przekazywanie informacji o ocenach studentom, a studentom – uzyskanie jasnej informacji o zaliczeniach etapów oraz ocenach cząstkowych. Oprogramowanie posiada funkcję zarządzania kontami studentów, prowadzących i administratorów oraz zarządzania ocenami z poziomu konta prowadzącego. Intuicyjny, łatwy w obsłudze interfejs jest jednym z kluczowych celów projektu.

1.3. Zakres pracy, architektura systemu, technologie

Warstwa logiki aplikacji obejmuje serwer, który zarządza logiką biznesową, współpracuje z bazą danych i komunikuje się z interfejsem (front-endem), dostępnym za pośrednictwem przeglądarki internetowej. Ponadto utworzona baza danych, przechowuje wszystkie informacje niezbędne do prawidłowego działania aplikacji.

JavaScript [1] został wybrany jako język programowania ze względu na dynamiczne typowanie, które przyspiesza rozwój aplikacji i zwiększa jej elastyczność. Jego biblioteka React [2] została wybrana do tworzenia front-endu ze względu na jej szerokie zastosowanie, popularność oraz aktywną społeczność użytkowników, co ułatwia pozyskiwanie informacji i wsparcia technicznego. React oferuje bogatą bibliotekę komponentów oraz wiele funkcjonalności, co umożliwia płynną integrację z backendem.

Dla warstwy backendowej wybrano framework Django [3], napisany w języku Python. Django ułatwia szybkie tworzenie skalowalnych aplikacji internetowych. Jedną z jego zalet jest wbudowany panel administracyjny, który ułatwia zarządzanie danymi i użytkownikami. Ponadto Django oferuje szerokie wsparcie w zakresie konfiguracji i integracji z bazami danych.

Django działa jako backend API, zapewniając separację logiki aplikacji od warstwy prezentacji. Dzięki temu frontend, napisany w React, działa niezależnie i komunikuje się z backendem za pośrednictwem REST API [4].

Zarówno Django, jak i React to technologie zyskujące na popularności i cenione za swoją wydajność oraz skalowalność. Oba narzędzia ułatwiają obsługę wielu użytkowników i efektywne zarządzanie złożonymi interfejsami użytkownika.

Aplikacja korzysta z PostgreSQL [5] jako systemu zarządzania bazą danych. PostgreSQL to solidny system o otwartym kodzie źródłowym, oferujący wszechstronną funkcjonalność, w tym obsługę zaawansowanych zapytań i transakcji. Oferuje także mechanizmy autoryzacji i uwierzytelniania użytkowników, co zapewnia bezpieczeństwo dostępu zarówno do aplikacji, jak i przechowywanych danych.

System kontroli wersji w projekcie oparto na Git [6], a kod przechowywano w repozytorium na GitHub [7].

1.4. Przegląd istniejących rozwiązań

Na rynku oprogramowań wspierających edukację istnieje wiele podobnych rozwiązań oferujących różnorodne funkcje. Wśród nich na szczególną uwagę zasługuje powszechnie stosowane środowisko do nauczania zdalnego *Moodle*, z którego według

danych twórców korzystało ponad 393 milionów użytkowników, zapisując się 2,3 miliarda razy na 46 milionów kursów [8]. Rozwiązanie to charakteryzuje się dostępnością w modelu open source oraz szerokim zakresem możliwości, dopasowanym do potrzeb klientów. W szczególności wyróżnia się zarządzaniem kursami i ocenami, co jest istotne w kontekście tej pracy. Użytkownikami platformy są szkoły wyższe, średnie i zawodowe, instytucje publiczne, sektor opieki zdrowotnej oraz firmy IT.

Kolejnym godnym uwagi rozwiązaniem jest *SchoolStatus* [9], które koncentruje się głównie na komunikacji między szkołą a rodzinami. Platforma integruje informacje o postępach uczniów, ich ocenach, obecnościach na zajęciach oraz osiągnięciach, przedstawiając je w przejrzystym, wizualnym formacie. SchoolStatus oferuje łatwe w użyciu narzędzia analityczne i komunikacyjne, umożliwiające bezpośredni kontakt z rodzinami uczniów. Celem tego oprogramowania jest zwiększenie efektywności i wyników uczniów oraz szkół, a także poprawa frekwencji dzięki skutecznej komunikacji.

2. Oprogramowanie

2.1. Wymagania

Aplikacja jest dostępna dla użytkowników końcowych w momencie spełnienia podstawowych wymagań: wystarczy komputer lub urządzenie mobilne z dostępem do internetu, przeglądarka internetowa oraz znajomość adresu URL aplikacji: `https://unigradeflow-backend.onrender.com/`. Dzięki temu korzystanie z systemu nie wymaga instalacji dodatkowego oprogramowania ani konfiguracji technicznej.

2.1.1. Środowisko deweloperskie

Proces deweloperski odnosi się do cyklu tworzenia, testowania i rozwijania aplikacji. W przypadku tego projektu oznacza to m.in. pisanie kodu frontendowego w React oraz backendowego w Django, uruchamianie środowiska lokalnego i testowanie nowych funkcji przed wdrożeniem na serwer.

Aby uruchomić aplikację lokalnie w celach deweloperskich, konieczne jest odpowiednie skonfigurowanie środowiska. Wymagane są:

- **Node Package Manager (npm)** [10]
- **Python 3** [11]
- **pip**: menedżer pakietów Pythona [12]
- **SQLite3**: silnik bazy danych [13]

Po zainstalowaniu wymaganych pakietów można uruchomić aplikację lokalnie, wpisując w terminalu:

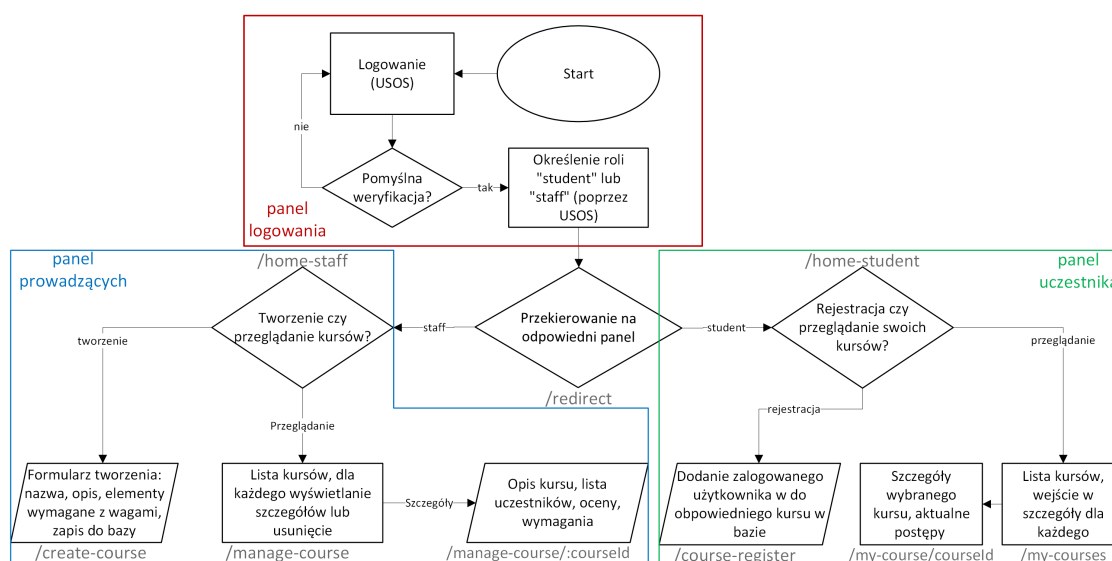
```
python manage.py runserver  
npm run dev
```

Te polecenia uruchomią backend pod adresem `http://127.0.0.1:8000/` oraz frontend (interfejs użytkownika) pod `http://localhost:5173/`. Po otwarciu tych adresów w przeglądarce można zobaczyć odpowiednio: ścieżki API backendu oraz interfejs aplikacji React.

2.2. Korzystanie z aplikacji

Aplikacja składa się z trzech kluczowych komponentów pod kątem funkcjonalności: panelu logowania, panelu prowadzących oraz panelu uczestnika. Dodatkowo zawiera elementy składające się zarówno z podsekcji dla prowadzących, uczestników, jak i niezależnych modułów. W każdym widoku strony stale wyświetlane są nagłówki (u góry) oraz stopka (na dole).

Poniższy diagram przedstawia schemat poruszania się po poszczególnych komponentach.



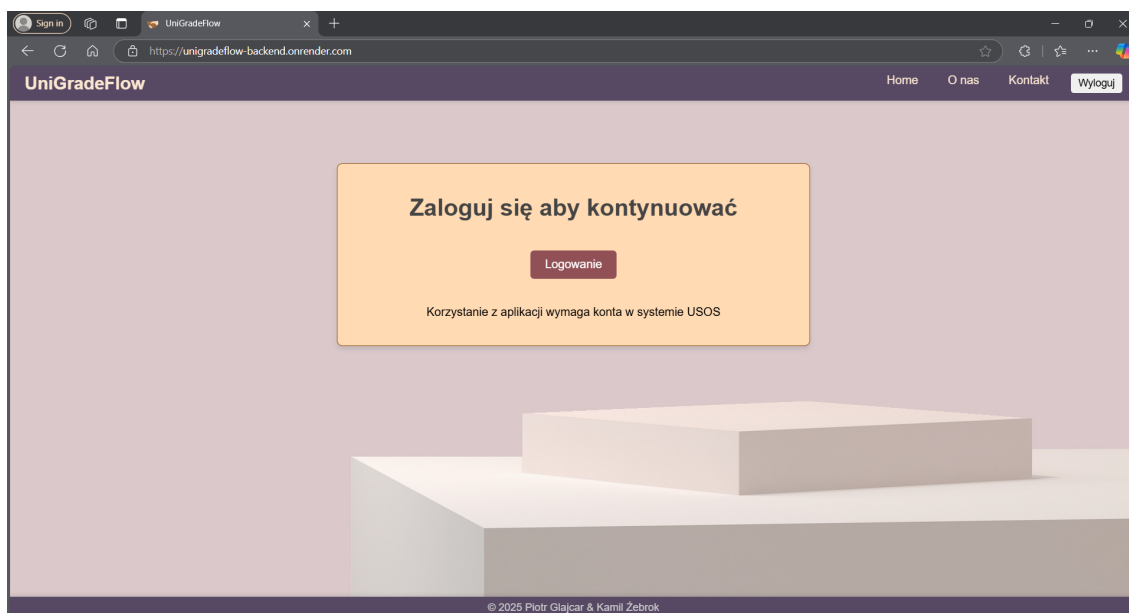
Rysunek 1: Schemat poruszania się po głównych komponentach

2.2.1. Start i logowanie

Po uruchomieniu aplikacji użytkownik trafia na stronę powitalną, gdzie ma możliwość zalogowania się lub przejścia do sekcji informacyjnych: „O nas” i „Kontakt”. Proces logowania i autoryzacji odbywa się za pośrednictwem USOS API, o którym więcej można przeczytać w rozdziale 4.5 dotyczącym autoryzacji.

W przypadku nieudanego logowania użytkownik pozostaje w centralnym serwisie uwierzytelniania (Central Authentication Service) USOS-a. Próba dostępu do innych komponentów niż wymienione sekcje informacyjne skutkuje przekierowaniem na stronę informującą o braku autoryzacji.

Po pomyślnym zalogowaniu system rozpoznaje rolę użytkownika i na tej podstawie przekierowuje go do panelu prowadzących kursy lub panelu studentów.

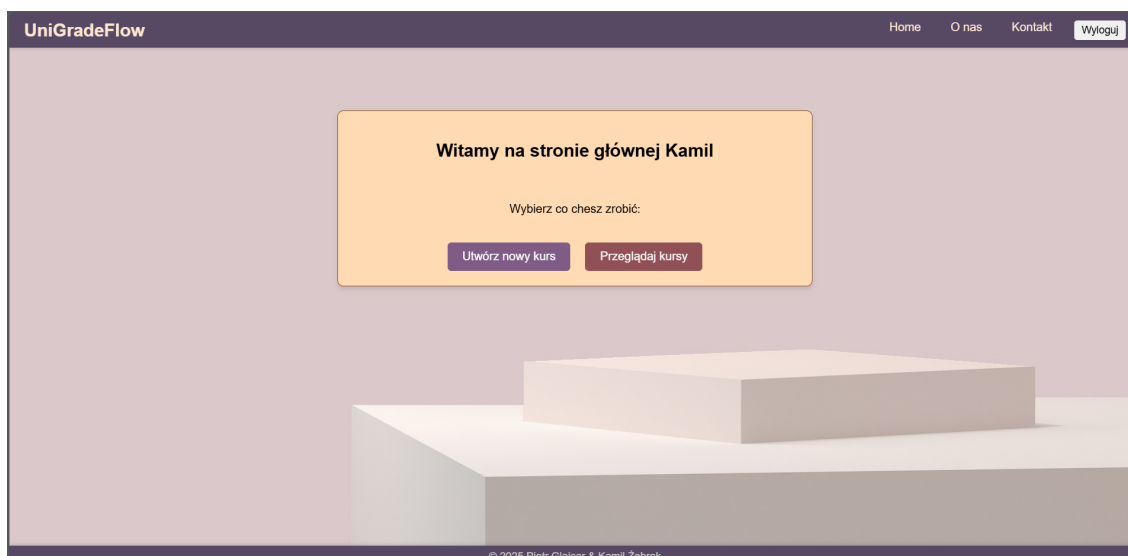


Rysunek 2: Ekran logowania

2.2.2. Panel prowadzących

Po wejściu do panelu prowadzących użytkownik widzi układ strony podobny do ekranu logowania, lecz zamiast formularza wyświetlana jest wiadomość powitalna zawierająca jego imię. Ponadto dostępne są dwa przyciski umożliwiające wybór jednej z dwóch opcji:

- **Tworzenie kursów** – przejście do formularza tworzenia
- **Przeglądanie kursów** – przejście do sekcji zawierającej listę utworzonych przez prowadzącego kursów.



Rysunek 3: Panel prowadzącego

Formularz tworzenia kursu składa się z pięciu pól tekstowych podzielonych na dwie sekcje:

- **Sekcji informacyjnej** – zawierającej pola na nazwę i opis
- **sekcji elementów wymaganych** – umożliwia określenie zadań koniecznych do zaliczenia kursu, takich jak egzamin czy projekt. W tej sekcji użytkownik wprowadza nazwę zadania, jego opis oraz wagę.

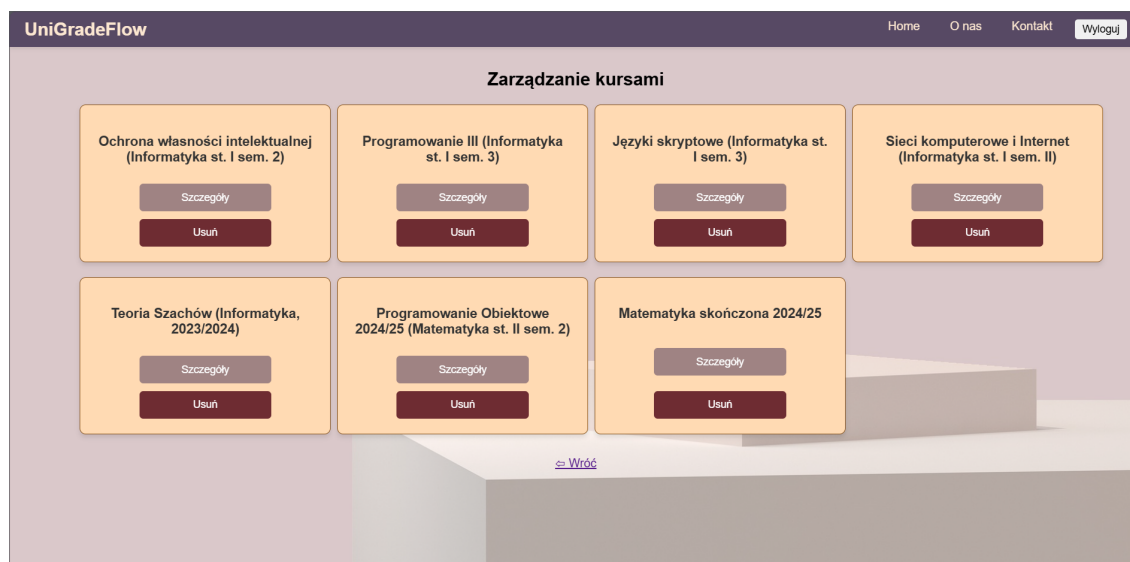
Pod sekcją elementów wymaganych znajduje się przycisk „**Dodaj element**”, który pozwala na dodanie kolejnych zadań wraz z ich wagą. Zadania te będą wyświetlane studentom, aby po rejestracji mogli śledzić swoje postępy w kursie.

Po poprawnym dodaniu nowego elementu pola w tej sekcji zostają wyczyszczone, a zadanie pojawia się poniżej przycisku. Kolejne dodawane elementy są dopisywane w formie listy.

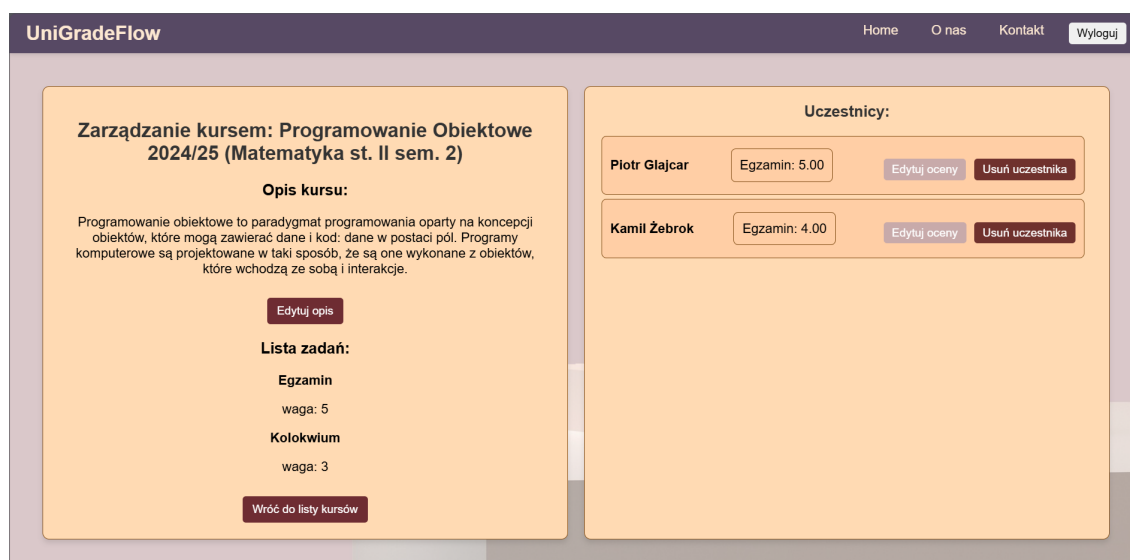
Na samym dole formularza, w celach informacyjnych, widnieje imię i nazwisko aktualnie zalogowanego użytkownika jako prowadzącego kurs. Obok znajduje się przycisk „Utwórz kurs”, który finalizuje proces tworzenia.

Rysunek 4: Formularz tworzenia kursu

Przeglądanie kursów, a właściwie zarządzanie nimi, to sekcja strony zawierająca listę wszystkich kursów utworzonych przez zalogowanego edukatora. Są one wyświetlane w formie „kafelków”, które układają się w zależności od ich ilości i rozmiarów ekranu, sprawiając, że całość jest poukładana i przejrzysta. Każdy kafelek jest związany z kursem, którego nazwa jest na nim wyświetlona i zawiera dwa przyciski, umożliwiające przejście do szczegółów kursu lub usunięcie kursu. Wybranie pierwszej z tych opcji spowoduje wejście w stronę wyświetlającą detale danego kursu: opis i listę zadań po stronie lewej, a listę uczestników z ich ocenami po prawej. Część z lewej to te dane, które zostały nadane kursowi w procesie tworzenia, lecz jedynie opis zawiera możliwość edycji, przez co zmiana nazwy i obligatoryjnych elementów wymaganych do zaliczenia kursu w trakcie jego przebiegu jest niemożliwa, a sam opis kursu może pełnić dodatkowo funkcję forum aktualności. Lista zarejestrowanych uczestników na kurs po prawej stronie wyświetla wylistowane oceny dla każdego uczestnika z osobna, a obok przyciski do ich edycji i usunięcia wskazanego kursanta. Ten pierwszy, po wciśnięciu, rozwija menu edycji ocen, w którym można rozwinąć listę z ocenianymi elementami i wpisać odpowiednią wartość liczbową, dodając i zapisując ją odpowiednimi przyciskami pojawiającymi się na spodzie.



Rysunek 5: Przeglądanie kursów



Rysunek 6: Szczegóły wybranego kursu

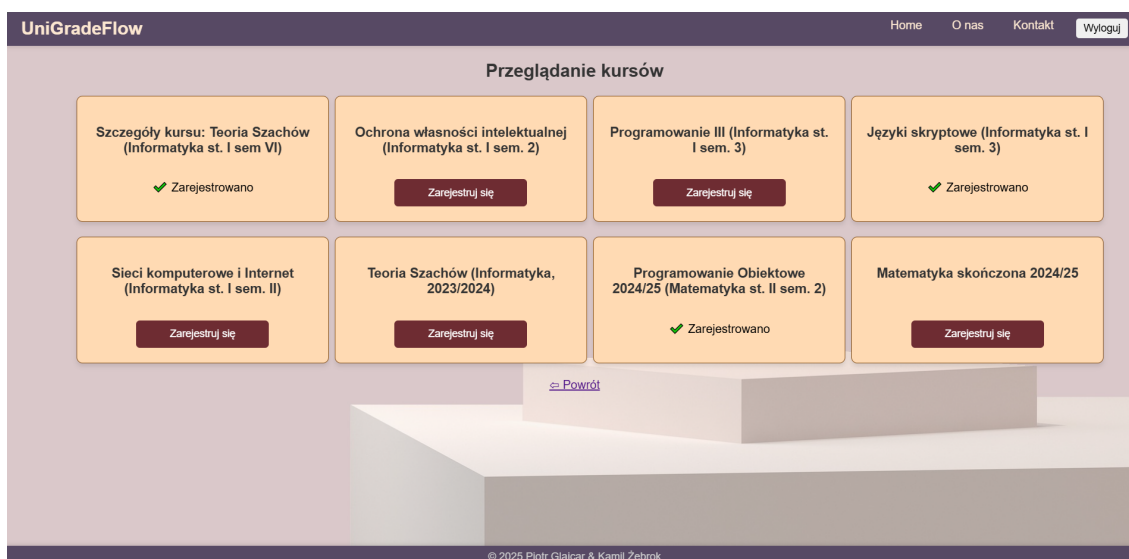
2.2.3. Panel uczestników

Jeżeli po przekierowaniu USOS Api określi rolę użytkownika jako **student**, aplikacja ukazuje stronę powitalną studenta. Pod nagłówkiem „Witaj, [Imię Nazwisko]”, podobnie jak w panelu prowadzących, są to dwa przyciski, z których pierwszy z lewej służy przejściu do witryny z **rejestracją na kursy**, a kolejny przeznaczony jest do wejścia w **przeglądanie kursów**, na które zalogowany uczestnik się uprzednio zarejestrował. Dodatkowo poniżej wyświetlane są e-mail i nr albumu studenta, umożliwiające proste zaznaczenie i skopiowanie tych danych, co może okazać się dużą pomocą i wygodą dla nowych studentów.



Rysunek 7: Strona powitalna kursanta

Rejestracja na kursy to strona o podobnej zawartości do sekcji zarządzania kursami prowadzącego, z taką różnicą, że wyświetlone są wszystkie kursy każdego z prowadzących i zamiast wcześniejszych przycisków, na „kafelkach” jest tylko jeden służący rejestracji. W momencie obsłużenia tego przycisku zamienia się on w napis „Zarejestrowano”, potwierdzający pomyślne zapisanie się studenta na wybrany kurs.



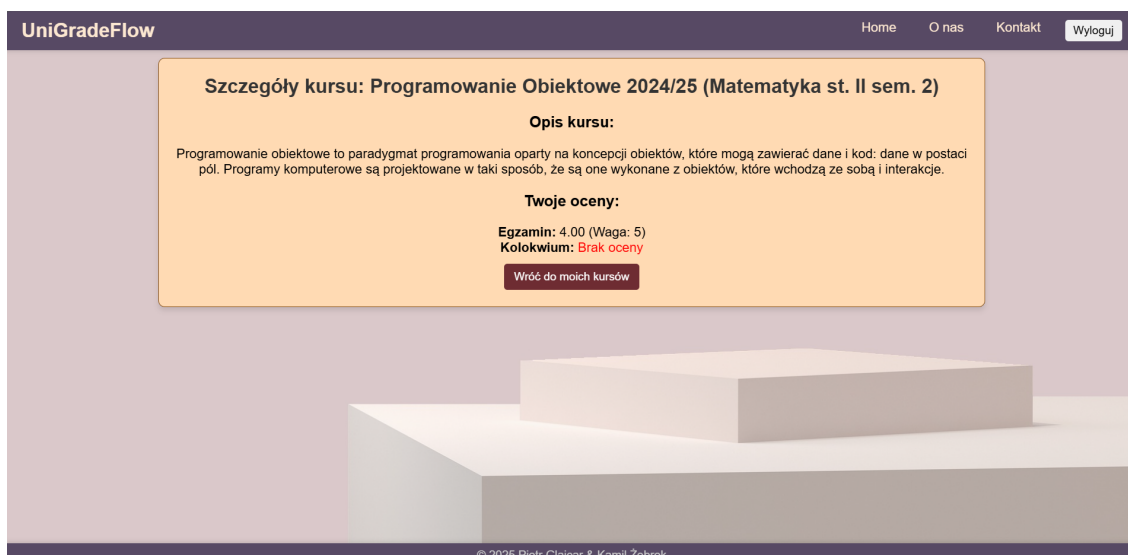
Rysunek 8: Panel rejestracji

Opcja **przeglądanie swoich kursów** wyświetla analogiczną stronę, gdzie, w kontraście do poprzedniej, ukazują się tylko kursy, w których student się znajduje, i każdemu z nich towarzyszy przycisk przejścia do szczegółów.

Na stronie szczegółów widnieje opis kursu oraz najważniejsza funkcjonalność strony, będąca inspiracją tego projektu – informacja o postępach studenta w danym kursie w formie listy obowiązkowych zadań, z wyraźnym przedstawieniem odpowiednich ocen lub ich braku. Wszystkie zadania są wyświetlane niezależnie od tego, czy student posiada z nich ocenę, a brak oceny jest wyróżniony czerwonym kolorem, uwypuklając te zadania, które kursant musi zdać, aby zaliczyć przedmiot.



Rysunek 9: Przeglądanie kursów, na które student się zarejestrował



Rysunek 10: Szczegóły wybranego przez studenta kursu

3. Interfejs Użytkownika

3.1. Narzędzia

Wszystkie elementy widoczne dla użytkownika, wraz z tymi, które umożliwiają jego interakcję ze stroną, zostały utworzone przy pomocy języka JavaScript oraz bibliotek React i React DOM [14]. Umożliwiają one tworzenie dynamicznych aplikacji webowych dzięki komponentowej architekturze, co pozwala na proste zarządzanie stanem aplikacji oraz wielokrotne wykorzystanie kodu.

W projekcie szczególnie często wykorzystywane są następujące funkcje i komponenty:

Z biblioteki React:

- **useEffect** – pozwala na wykonywanie dodatkowych operacji po każdym renderze (generowaniu i wyświetlaniu) komponentu, np. pobieranie danych lub reakcję na zmiany.
- **useState** – służy do przechowywania i aktualizacji danych w komponencie.
- **useContext** – umożliwia udostępnianie danych wielu komponentom.

Wszystkie trzy są zwane „hookami”, czyli specjalnymi funkcjami, które ułatwiają pisanie kodu i zarządzanie logiką wielokrotnego użytku w funkcyjnych komponentach, upraszczając kod i poprawiając czytelność [15].

Z biblioteki React Router (react-router-dom):

- **useNavigate** (hook) – umożliwia zmianę strony bez konieczności przeładowania, przez deklarowane przekierowanie.
- **Navigate** (komponent) – automatycznie przekierowuje użytkownika na inną stronę, wywoływany przez zdarzenia lub logikę.
- **useParams** – pozwala na odczytanie parametrów z adresu URL, np. identyfikatora użytkownika.

Podstawowa struktura tego projektu powstała przy pomocy lokalnego serwera deweloperskiego Vite [16], obsługującego JSX (JavaScript XML) – format zapisu

kodu HTML oraz XML wewnątrz JavaScript, w którym zapisana jest większość komponentów interfejsu projektu.

Stylizacja aplikacji została zrealizowana głównie poprzez plik CSS, który definiuje układ, odstępy i responsywność strony. Utworzenie odpowiedniej liczby komponentów w JSX, nadanie im stosownego wyglądu, ich integracja z logiką backendu i bazy danych, a następnie ich wdrożenie stanowi realizację celu i założeń pracy.

3.2. Style i kolorystyka

W stylizacji aplikacji kluczową rolę odgrywa plik CSS, który odpowiada za większość widocznych elementów interfejsu. Definiuje on m.in. układ, odstępy oraz responsywność, wpływając na końcowy wygląd i wygodę użytkownika.

Kolorystyka strony została zainspirowana paletą znalezioną na stronie Color Hunt [17] i zmodyfikowana dla kilku elementów strony. W tle strony dominuje zgaszony pudrowy róż z nutą beżu, dodający miękkości i spokoju, wraz z jasnymi bryłami usytuowanymi w prawym dolnym rogu ekranu.

Najważniejsza treść strony, zawierająca wszystkie funkcjonalności, znajduje się w wydzielonym fragmencie w ciepłym odcieniu brzoskwiniowym, a przyciski w nim zawarte są w odcieniach fioletowych i ceglanej czerwieni. Paski nawigacyjne na górze i na dole ekranu, w kolorze ciemnego fioletu z odrobiną szarości, dodają elegancji i kontrastu, podkreślając bardziej stonowane barwy w paletce.

Wszystkie te kolory zostały umieszczone w odpowiednich klasach w pliku `index.css` w formacie parametrów RGBA [18].

3.3. Struktura plików związanych z interfejsem

Pliki interfejsu znajdują się w folderze `frontend`, aby odseparować je od plików związanych z logiką aplikacji, które omawiane są szczegółowo w rozdziale 4.

Pierwotna struktura tego folderu została zainicjowana komendą `npm init vite`, po której wybrano model (framework) `React + JavaScript`, generując odpowiednie pliki:

<code>frontend/</code>	<code># Katalog interfejsu użytkownika</code>
<code> public/</code>	<code># Pliki statyczne</code>
<code> favicon.ico</code>	<code># Ikona karty (zakładki) - favicon</code>
<code> src/</code>	<code># Kod źródłowy aplikacji</code>
<code> assets/</code>	<code># Folder na zasoby statyczne obrazy, tło</code>

```

App.jsx      # Główny komponent aplikacji
main.jsx     # Punkt wejściowy aplikacji
index.css    # Globalne style
.gitignore   # Plik ignorujący pliki w repozytorium Git
eslint.config.js # Konfiguracja narzędzia ESLint
index.html   # Główny plik HTML aplikacji
package.json # Plik konfiguracyjny npm (zależności, skrypty)
README.md    # Plik z podstawowymi informacjami o projekcie
vite.config.js # Konfiguracja Vite

```

Należy wyróżnić kluczowe pliki projektu, w tym również te dodane w późniejszej fazie jego rozwoju, wraz z ich modyfikacjami:

- `src/main.jsx` – punkt wejściowy aplikacji, renderuje `App.jsx` do DOM (obiektowego modelu dokumentu), czyli sposobu reprezentacji dokumentu HTML zgodnego z paradygmatem obiektowości [19].
- `src/App.jsx` – główny komponent aplikacji, pochodzący z szablonowego modułu `React + Vite`, zmieniony w mapę nawigacji po każdym z widoków, z wykorzystaniem `React Router`.
- `index.html` – główny plik w języku HTML, do którego Vite montuje aplikację po dodaniu skryptu `<script type="module"src="/src/main.jsx"/>` w `<body>`.
- `vite.config.js` – plik konfiguracji Vite, odpowiedzialny za ustawienia projektowe, takie jak alternatywne ścieżki (aliasy), rozszerzenia dodające dodatkowe funkcjonalności (pluginy) oraz przesyłanie żądań między frontendem a backendem (proxy) [20].

W początkowej i finalnej wersji jedynym używanym pluginem jest `react()`, który umożliwia obsługę JSX i optymalizację pod kątem `Reacta`. Na etapie pracy w środowisku deweloperskim istotne było skonfigurowanie proxy dla żądań związanych z autoryzacją, aby przekierować je na odpowiedni adres backendu (`http://localhost:8000`).

- `components/` – folder na komponenty, czyli wszystkie pojedyncze pliki zasobów różnych sekcji strony.

Ostatni punkt z powyższej listy zawiera 18 plików typu JSX. Każdemu z nich, z wyjątkami (m.in. nagłówkiem i stopką) przypisana jest odpowiednia ścieżka w pliku `App.jsx` przy pomocy `React Router`.

3.3.1. App.jsx

Ten plik stanowi o tym, co znajdzie się w wybranych ścieżkach aplikacji. Jego uproszczoną budowę można przedstawić w następujący sposób:

frontend/App.jsx

```
function App() {  
  return (  
    <Router>  
      <Header />  
      <Routes>  
        <Route path="/ścieżka1" element={<NazwaElementu1 />} />  
        <Route path="/ścieżka2" element={<NazwaElementu2 />} />  
        ...  
      </Routes>  
      <Footer />  
    </Router>  
  )  
}
```

Aby zachować spójność i uzyskać pożądany widok, nagłówek i stopka znajdują się poza ścieżkami, a ponadto dodano tagi `<div>` pomiędzy niektórymi elementami, umożliwiające ustalenie stylu po nadaniu nazw klas tym tagom.

W JavaScript można eksportować wartości, funkcje, klasy lub obiekty metodą `export default`, aby były dostępne w innych modułach, i ta koncepcja została zastosowana do wszystkich 18 plików folderu `components`. W wyniku tego podejścia każdy z nich kończy się liniijką `export default NazwaPliku`, a w głównym komponencie aplikacji, po dodaniu `import NazwaPliku from './components/NazwaPliku.jsx'` i umieszczeniu go w funkcji `App` w formacie `<NazwaPliku />`, są one poprawnie włączone do aplikacji.

Komponenty w tym miejscu podzielone są na ścieżki publiczne (dostępne bez konieczności autoryzacji), ścieżki pomocnicze (przekierowanie, wylogowanie, zablokowanie dostępu), ścieżki kursantów oraz ścieżki prowadzących.

3.3.2. ProtectedRoute.jsx

Istotnym aspektem poprawnego działania aplikacji jest również to, że część ścieżek powinna być zabezpieczona, ponieważ niektóre aktywności są przeznaczone tylko dla określonych użytkowników. Rozwiązaniem tej funkcjonalności jest komponent

`ProtectedRoute.jsx`, który zawiera w kodzie instrukcje warunkowe przekierowujące użytkownika odpowiednio do ekranu logowania w przypadku braku zalogowanego użytkownika (`!user`) oraz do strony `Unauthorized.jsx` w przypadku, gdy rola użytkownika nie zgadza się z przypisaną do komponentu (`user.role !== role`). Poprawna implementacja tych zabezpieczeń następuje poprzez umieszczenie członu `<ProtectedRoute role='nazwa-rola'><NazwaElementu /></ProtectedRoute>` w danej ścieżce pliku `App.jsx` przypisując ją jako wartość cechy `element`.

```
<Route path="/my-course/:courseId" element={
  <ProtectedRoute role="student">
    <MyCourseDetails />
  </ProtectedRoute> }/>
```

Rysunek 11: Przykład zastosowania `ProtectedRoute`

3.3.3. Logowanie, ścieżki publiczne i pomocnicze

Aby zminimalizować zakres zbieranych danych od użytkowników oraz uniknąć konieczności uzyskiwania dodatkowych zgód związanych z ochroną danych osobowych, aplikacja korzysta z logowania za pośrednictwem Uniwersyteckiego Systemu Obsługi Studiów (USOS). USOS oferuje otwarte, dobrze rozwinięte API, zapewniające wysoki poziom bezpieczeństwa wrażliwych danych użytkowników. Szczegóły dotyczące integracji z USOS od strony backendu zostały opisane w rozdziałach 4.7 i 4.8.

Mechanizm przekierowania do systemu USOS został zaimplementowany w komponencie `ToLogin.jsx`. Użytkownik rozpoczyna proces logowania poprzez kliknięcie przycisku, który wywołuje funkcję obsługującą przekierowanie. Funkcja ta dynamicznie generuje adres URL do elementu związanego z autoryzacją na backendzie aplikacji, korzystając ze zmiennych środowiskowych. Adres ten jest tworzony w sposób przedstawiony na rysunku 12.

```
const handleLogin = async () => {
  window.location.href = `${import.meta.env.VITE_API_BASE_URL}/oauth/start/`;
};
```

Rysunek 12: Dynamiczne generowanie adresu URL z użyciem zmiennych środowiskowych

Funkcja `handleLogin` wykorzystuje słowo kluczowe `async` do definiowania funkcji asynchronicznej w JavaScript. Oznacza to, że funkcja może wykonywać swoje zadanie bez blokowania reszty kodu i natychmiast zwraca obietnicę (promise), czyli

zastępczy wynik, który będzie dostępny później, kiedy odpowiednio się załaduje (zostanie rozwiązany lub odrzucony) [21]. Jest ona również używana w wielu innych komponentach aplikacji, szczególnie przy pobieraniu i wysyłaniu danych do bazy.

Zmienne środowiskowe pozwalają na elastyczne dostosowanie adresu backendu w zależności od środowiska uruchomieniowego (lokalnego lub produkcyjnego). Po wygenerowaniu odpowiedniego adresu rozpoczyna się proces, w którym użytkownik zostaje przekierowany na stronę logowania USOS, gdzie po pomyślnym uwierzytelnieniu kończy proces autoryzacji i zostaje przekierowany do komponentu `RedirectPage.jsx`.

Komponent ten pełni rolę tymczasowej strony, której głównym zadaniem jest sprawdzenie poprawności danych użytkownika oraz przekierowanie go do wskazanego widoku w zależności od przypisanej roli. Nie powinien on być długo wyświetlany, ponieważ natychmiast po załadowaniu wykonuje odpowiednie przekierowanie, więc brak jego zmiany może oznaczać problem z połączeniem internetowym lub inną awarię.

Działanie komponentu opiera się na funkcji biblioteki React `useEffect`, która uruchamia funkcję sprawdzającą dane użytkownika. Jeśli dane nie są jeszcze załadowane, komponent wstrzymuje dalsze operacje. W przypadku braku informacji o użytkowniku następuje przekierowanie na stronę logowania. Jeśli użytkownik jest zalogowany, system sprawdza jego rolę i kieruje go do odpowiedniego interfejsu:

- `/home-student` – jeśli użytkownik ma rolę studenta,
- `/home-staff` – jeśli użytkownik ma rolę nauczyciela,
- `/unauthorized` – jeśli rola użytkownika nie jest rozpoznana.

Podczas tego procesu komponent zwraca jedynie prosty komunikat informujący użytkownika o ładowaniu strony i sprawdzaniu jego uprawnień. W praktyce komunikat ten powinien być widoczny jedynie przez krótki czas, zanim nastąpi właściwe przekierowanie.

Ostatni podpunkt powyższej listy, czyli `/unauthorized`, jest elementem publicznym, podobnie jak komponenty `About`, `Contact` i `LoggedOut`, dostępne w pasku nawigacyjnym. Pełnią one wyłącznie funkcję informacyjną i nie pozwalają na interakcję z aplikacją ani na dostęp do danych użytkowników. Obok nich w nagłówku widnieje również `Home`, który realizuje przekierowanie do `RedirectPage`, więc bez zalogowania również dołącza się do tej grupy elementów.

`UserContext` jest kluczowym komponentem, który umożliwia klarowny i wielokrotnego użytku sposób udostępniania informacji o użytkowniku w całej aplikacji.

Poza kilkoma modułami z React, używa również biblioteki `js-cookie`, która umożliwia łatwe zarządzanie, pobieranie i manipulowanie plikami cookie przeglądarki. Jest to szczególnie przydatne do obsługi zadań związanych z uwierzytelnianiem. Komponent `UserContext` zarządza stanem uwierzytelniania użytkownika, w tym pobieraniem danych bieżącego użytkownika, obsługą funkcji wylogowania i zapewnianiem, że sesja użytkownika jest prawidłowo utrzymywana.

Jeśli niezalogowany użytkownik spróbuje uzyskać dostęp do chronionych zasobów poprzez ręczne wpisanie adresu URL, aplikacja automatycznie przekieruje go na stronę `/unauthorized`. W przypadku wpisania nieprawidłowego adresu w ścieżce URL zostanie natomiast wyświetlona strona z nagłówkiem, pustą zawartością (tłem) i stopką.

3.3.4. Strony główne użytkowników

Zalogowanie i właściwe przekierowanie prowadzi użytkownika do komponentu `Home` lub `HomeStudent`. Tego pierwszego nie należy mylić z przyciskiem „Home” w pasku nawigacyjnym, który przenosi w odpowiednie miejsce w zależności od roli. Plik `Home.jsx` definiuje stronę główną dla użytkowników z rolą `teacher`. Jego głównym zadaniem jest wyświetlenie dynamicznego komunikatu powitalnego, zawierającego imię użytkownika pobrane z `UserContext`, który jest importowany do tego pliku. Dodatkowo komponent umożliwia dostęp do funkcji zarządzania kursami oraz tworzenia nowych kursów.

- Struktura komponentu obejmuje elementy `div` opatrzone klasami CSS, które służą do:
 - wyśrodkowania zawartości na stronie,
 - odpowiedniego rozmieszczenia przycisków,
 - nadania im estetycznych stylów oraz efektów wizualnych (np. zmiana koloru po najechaniu kursorem).
- Wykorzystuje mechanizm nawigacji `useNavigate()` z biblioteki React Router, którym dwa główne przyciski nawigacyjne przekierowują użytkownika do:
 - strony tworzenia kursów (`/create-course`),
 - strony przeglądania istniejących kursów (`/manage-course`).

Plik `HomeStudent.jsx` zawiera stronę główną dla studentów (użytkowników z rolą `student`). Jego funkcjonalność oraz użycie klas CSS są podobne do `Home.jsx`, jednak są również dostosowane do specyfiki ról użytkowników – komponent umożliwia

rejestrację na nowe kursy oraz przeglądanie już przypisanych użytkownikowi przedmiotów. Dzięki temu zapewnia spójność wizualną interfejsu, lecz ze względu na różne dostępne funkcjonalności komponent ten wymaga jednak osobnej implementacji.

Jeśli użytkownik jest poprawnie zalogowany, wyświetlany jest dynamiczny komunikat powitalny zawierający imię i nazwisko, a także, w odróżnieniu od nauczyciela, dodatkowe informacje, takie jak adres e-mail oraz numer albumu, które są pobierane z `UserContext`.

```
<h1>Witaj, {user.first_name} {user.last_name}!</h1>
<div className="buttons">
  <button className="button" onClick={() => navigate("/course-register")}>Zarejestruj się na nowy kurs</button>
  <button className="button" onClick={() => navigate("/my-courses")}>Przeglądaj swoje kursy</button>
</div>
<p>email: {user.email}
  <br></br>
  nr albumu: {user.student_number}
</p>
```

Rysunek 13: Powitanie z dynamicznym pobieraniem i wyświetlaniem danych użytkownika oraz przyciski nawigujące w `HomeStudent.jsx`

Dwa główne przyciski umożliwiają przejście do rejestracji na nowy kurs, przekierowując użytkownika na stronę `/course-register`, oraz przeglądanie już zapisanych kursów poprzez nawigację do `/my-courses`.

3.3.5. Moduł prowadzących – dostępne ścieżki i funkcjonalności

Dla użytkowników posiadających rolę `teacher`, poza komponentem `Home`, w aplikacji przewidziano trzy dedykowane ścieżki: `CreateCourse`, `ListCourses` oraz `ManageCourse`. Każda z nich odpowiada za inną część procesu tworzenia, przeglądania oraz zarządzania kursami i ocenami.

Tworzenie kursów, które mają trafić do bazy danych, odbywa się poprzez poprawne wypełnienie formularza zaimplementowanego w pliku `CreateCourse.jsx`. Jest to standardowy dynamiczny formularz React z polami `<input />` dla docelowo krótszych tekstów, takich jak nazwa kursu, nazwy wymaganych elementów i ich wagi, oraz z polami `<textarea />` dla opisów kursów i wymagań. Każde takie pole obarczone jest odpowiednią etykietą `<label>` i zawiera swój rodzaj zmiennej, atrybut `value`, który sprawia, że wartość pola odzwierciedla jej bieżącą wartość oraz funkcję obsługi zdarzenia `onChange`. Zmienne utworzone i zarządzane są przy użyciu hooka `useState()` w następujący sposób:

```
const [nazwaElementu, setNazwaElementu] = useState("");
```

Wartość początkowa przekazana do `useState()` zależy od przechowywanych danych – może to być pusty ciąg znaków ("") dla tekstu, pusta tablica ([]) dla listy elementów lub liczba (1) dla wartości numerycznych (np. wagi oceny). Dzięki temu możliwe jest przypisanie funkcji aktualizującej wartość `nazwaElementu` do atrybutu zdarzenia `onChange`:

```
onChange={(e) => setNazwaElementu(e.target.value)}
```

Pod formularzem znajdują się dwa przyciski – jeden do dodawania wymaganych elementów, a drugi do finalizacji tworzenia kursu. Między nimi wyświetlana jest lista dodanych elementów wymaganych (początkowo pusta i niewidoczna) oraz informacje o prowadzącym kurs, obejmujące imię, nazwisko i numer identyfikacyjny zalogowanego użytkownika. Identyfikator nauczyciela jest zapisywany w bazie danych, co umożliwia późniejsze filtrowanie kursów według prowadzących.

Atrybut `onClick` przycisku „Dodaj element” nazwany `handleAddRequiredElement` odpowiada za obsługę dodawania nowych elementów wymaganych do kursu. Funkcja ta weryfikuje poprawność danych wejściowych, a następnie aktualizuje stan aplikacji, dodając nowy element do listy.

Jeśli wartości `elementName` i `elementWeight` są poprawne (nazwa nie jest pusta, a waga jest większa od zera), funkcja tworzy nowy obiekt z nazwą, opisem i wagą elementu, a następnie dodaje go do tablicy `requiredElements` za pomocą funkcji `setRequiredElements`. Operator rozproszenia (`...requiredElements`) zapewnia, że do nowej tablicy zostaną dołączone wszystkie wcześniej dodane elementy, a nowy obiekt zostanie dopisany na końcu.

Po pomyślnym dodaniu elementu formularz jest resetowany – wartości `elementName`, `elementDescription` i `elementWeight` są ustawiane na wartości początkowe, co umożliwia użytkownikowi wprowadzenie kolejnego elementu.

W przypadku nieprawidłowych danych funkcja wyświetla komunikat ostrzegawczy za pomocą `alert()`, informując użytkownika o konieczności poprawnego uzupełnienia pól.

Przycisk **Utwórz kurs** wywołuje funkcję `handleCreateCourse`, która odpowiada za przesłanie danych nowego kursu do backendu oraz ich zapis w bazie danych.

Funkcja najpierw sprawdza, czy użytkownik wprowadził nazwę kursu oraz dodał przynajmniej jeden wymagany element. Jeśli te warunki są spełnione, tworzy obiekt `newCourse`, który zawiera:

- nazwę kursu (`courseName`),
- opcjonalny opis (`courseDescription`),

- listę wymaganych elementów (`writable_assignments`),
- identyfikator nauczyciela (`user.teacher_id`).

Następnie funkcja wykonuje żądanie POST do API pod adresem `/courses/?include=assignments`, przesyłając dane nowego kursu. Jeśli serwer zwróci kod 201 (co oznacza, że kurs został poprawnie utworzony), użytkownik otrzymuje powiadomienie o pomyślnym rezultacie, a formularz zostaje wyczyszczony. Po tym następuje przekierowanie na stronę zarządzania kursami (`/manage-course/`).

Jeśli serwer zwróci inny status lub wystąpi błąd (np. problem z połączeniem), użytkownik zostaje poinformowany o niepowodzeniu operacji za pomocą komunikatu `alert()`. Jeśli dane wejściowe są niekompletne, funkcja również wyświetla ostrzeżenie, aby użytkownik uzupełnił wymagane pola.

Za **zarządzanie pojedynczym kursem** odpowiada komponent `ManageCourse`. Umożliwia modyfikację jego opisu, przeglądanie listy uczestników, edycję ocen oraz dodawanie nowych elementów oceniania.

Po załadowaniu komponentu wykonywane jest żądanie GET do API pod adresem `/courses/courseId/?include=assignments&include=students`, które pobiera szczegółowe informacje o kursie: nazwę i opis, wymagania oraz listę zapisanych studentów wraz z ocenami i przechowuje je w zmiennej `selectedCourse`. Niektórymi danymi w tym komponencie można manipulować poprzez przyciski: „Edytuj opis” przy opisie, oraz „Edytuj oceny” i „Usuń uczestnika” przy każdym uczestniku. Lista uczestników kursu jest generowana dynamicznie na podstawie danych pobranych z API. Do jej wyświetlenia wykorzystywana jest metoda `map()`, która iteruje po tablicy `selectedCourse.students`, tworząc dla każdego studenta element listy. Jeśli kurs nie zawiera uczestników, zamiast listy pojawia się komunikat informujący o jej braku.

- Edycja opisu kursu: użytkownik może edytować opis kursu, który następnie zostaje zapisany w bazie danych poprzez żądanie PUT do API (`/courses/courseId/`).
 - Jeśli pole jest puste, wyświetlany jest komunikat ostrzegawczy.
 - Po zapisaniu zmian interfejs jest aktualizowany, a użytkownik otrzymuje powiadomienie.
- Edycja i dodawanie ocen: prowadzący może zmieniać oceny studentów z danych elementów wymaganych oraz specjalnego elementu „inne”, który dodaje elastycznej funkcjonalności dla niestandardowych sytuacji.

- Jeśli użytkownik chce edytować oceny studenta, komponent pobiera jego obecne oceny oraz identyfikator przypisania do kursu (`enrollment_id`).
 - Istniejące oceny są edytowane poprzez `PUT /grades/grade_id/`.
 - Nowe oceny są dodawane poprzez `POST /grades/`, a jeśli ocena dotyczy nowego zadania („inne”), to najpierw tworzony jest wpis w `/assignments/`.
- Usuwanie uczestnika: każdy uczestnik jest wyświetlany wraz z przyciskiem „Usuń uczestnika”, który po kliknięciu otwiera okno potwierdzenia. Po zatwierdzeniu operacji wykonywane jest żądanie `DELETE` do API, usuwające studenta z bazy danych. Następnie lista uczestników jest aktualizowana – usunięty student zostaje odfiltrowany ze stanu `selectedCourse`, a użytkownik otrzymuje komunikat o pomyślnym zakończeniu operacji.

Dzięki dynamicznej aktualizacji interfejsu usunięcie uczestnika, edycja opisu lub edycja ocen są natychmiast widoczne, a mechanizm potwierdzenia zapobiega przypadkowym błędom.

Przeglądanie kursów prowadzącego jest komponentem, który pobiera listę kursów z backendowego API, wyświetla je w widoku kafelkowym, by wyglądały atrakcyjnie i były dobrze zorganizowane, jak rozstawiona siatka kafelków z odstępami. Za taki widok odpowiada klasa `course-tile` zaimplementowana w pliku CSS, wykorzystywana również w module uczestników. Każdy kurs zawiera opcje przeglądania szczegółów kursu lub jego usunięcia, więc komponent ten uzyskuje funkcjonalności dopiero po utworzeniu przynajmniej jednego kursu, w przeciwnym wypadku wyświetlany jest paragraf o braku dostępnych kursów i przycisk powrotu do poprzedniej strony. Hook `useState` odpowiada za przechowywanie tablicy pobranych kursów w zmiennej `courses`, a `useEffect` do asynchronicznego pobierania kursów, kiedy komponent się montuje (czyli uruchamia funkcję `fetchCourses` po raz pierwszy). W tej samej funkcji zaimplementowane jest również filtrowanie na podstawie ID zalogowanego nauczyciela (`user?.teacher_id`) i aktualizacja stanu komponentu (`setCourses(teacherCourses)`).

```

useEffect(() => {
  const fetchCourses = async () => {
    try {
      const response = await backend.get("/courses/");

      const allCourses = Array.isArray(response.data.data) ? response.data.data : [];

      const teacherCourses = allCourses.filter(course => course.teacher === user?.teacher_id);

      setCourses(teacherCourses);
    } catch (error) {
      console.error("Error fetching courses:", error);
    }
  };

  fetchCourses();
}, [user]);

```

Rysunek 14: Asynchroniczne montowanie komponentu, filtrowanie kursów na podstawie ID nauczyciela i aktualizacja zmiennej `courses`.

3.3.6. Moduł uczestników – system przeglądania i rejestracji

Struktura części kursantów, z rolą `student` wykorzystuje wiele podobnych mechanizmów do tych z modułu prowadzących, lecz zawiera subtelne różnice związane z jej przeznaczeniem. Również dzieli się na trzy główne komponenty poza powitalnym `HomeStudent`: `CourseRegister`, `MyCourses` i `MyCourseDetails`.

`CourseRegister.jsx` odpowiadający za **rejestrację na kursy** aktualnie zalogowanego użytkownika umożliwia przeglądanie wszystkich dostępnych w bazie kursów i wybranie, do którego może się dopisać lub wskazać, że już jest jego uczestnikiem. Wcześniej używaniem `useEffect` wczytuje kursy (`courses`) i zapisy (`enrollments`), tym razem filtrując tylko zapisy po ID studenta. Dzięki temu podejściu istnieje możliwość wskazania, że student jest już zapisany na poszczególne kursy, a następnie też wyświetlić je w komponencie `MyCourses`. Obsługa rejestracji, po przyciśnięciu „Zarejestruj się” zachodzi w następujący sposób:

- Wywoływana zostaje asynchroniczna funkcja `handleRegister`, używająca ID kursu jako argument i weryfikująca ponownie czy użytkownik jest odpowiednio zalogowany
- Sprawdza, czy student jest już zapisany na kurs używając tablicy `enrollments`, w przypadku sukcesu informuje użytkownika wyświetlając odpowiedni komunikat.
- Jeśli w `enrollments` nie ma zapisu na wybrany przedmiot, wysyła do niego żądanie POST, aby zarejestrować studenta na kurs.

- Udana rejestracja kończy się powiadomieniem wyświetlanym u góry i znikającym po 3 sekundach (z użyciem `setTimeout`) i zmianą przycisku w napis „Zarejestrowano”.

Sprawdzenie, czy zalogowany użytkownik jest zarejestrowany na kurs, zachodzi z użyciem metody `some()` – wbudowanej metody tablic w JavaScript, która sprawdza, czy przynajmniej jeden element w tablicy spełnia podany warunek. Iteruje ona przez tablicę `enrollments` i jeśli jakikolwiek zapis w tablicy ma `course_id`, które odpowiada `course.id` kursu, którego przycisk został wybrany `some()` natychmiast zwraca `true`.

```
{courses.map((course) => {
  const isEnrolled = enrollments.some(enrollment => enrollment.course_id === course.id);

  return (
    <div key={course.id} className="course-tile">
      <h3>{course.name}</h3>

      {isEnrolled ? (
        <p className="registered-text">✔ Zarejestrowano</p>
      ) : (
        <button onClick={() => handleRegister(course.id)}>Zarejestruj się</button>
      )}
    </div>
  );
})}
```

Rysunek 15: Wyświetlanie kafelków kursów, użycie metody `some()` do wskazania kursów, do których student jest zarejestrowany

Udana rejestracja nie przenosi automatycznie do wybranego kursu, należy przejść do niego ręcznie poprzez przycisk „Przeglądaj swoje kursy” w poprzednim komponencie `HomeStudent`. Dzięki temu można rejestrować się na wiele kursów za jednym razem,

Przeglądanie swoich kursów zakodowane jest w `MyCourses.jsx`, korzysta z tych samych stylów (kafelki) i metod, zmodyfikowanych w taki sposób, by pobrane były jedynie zapisy zawierające ID zalogowanego studenta. Udane pobranie i przefiltrowanie danych odpowiednio ustawia zmienną `enrollments`, która następnie wykorzystuje `map()` do wyświetlenia kursów, jeżeli jest ich więcej niż 0. Jeżeli nie jest to spełnione, na stronie widnieje tekst:

Nie jesteś jeszcze zarejestrowany na żadne kursy.

Kafelki w tym komponencie zawierają nazwę kursu i przycisk „szczegóły”, służący do nawigowania do dynamicznej ścieżki aplikacji. Funkcja `navigate()` pobiera ją

w postaci `/my-course/$encodeURIComponent(enrollment.course_id)`, gdzie `encodeURIComponent` to wbudowana funkcja JavaScript, która koduje specjalne znaki w taki sposób, aby można je było bezpiecznie umieścić w URL. Ścieżka ta jest również odpowiednio zaimplementowana w głównym komponencie `App.jsx`, by prawidłowo udostępnić komponent `MyCourseDetails`:

```
<Route path="/my-course/:courseId" element={
  <ProtectedRoute role="student">
    <MyCourseDetails />
  </ProtectedRoute> }/>
```

Szczegóły kursu studenta zawarte w tym komponencie różnią się od pozostałych elementów modułu tym, że wyświetlają wszystkie detale związane z kursem i jego ocenami. Trzy razy użytymi hookami `useEffect` pobierane są:

- Szczegóły kursu, w tym zadań i studentów, na podstawie `courseId` z URL.
- Zapis studenta na kurs na podstawie `courseId` i ID zalogowanego studenta (`user.student_id`).
- Ocenę studenta za kurs na podstawie `enrollmentId`.

Załadowana poprawnie strona wyświetla nazwę kursu, opis oraz listę wymaganych zadań. Dla każdego zadania sprawdza, czy student ma ocenę i wyświetla ją, jeżeli jest dostępna, w przeciwnym razie pokazuje „Brak oceny”. Pary „wymaganie”-„ocena” (lub jej brak) są umiejscowione w liście nieuporządkowanej. Wybranie odpowiednich wymagań zaliczenia zachodzi metodą `map()` iterując przez tablicę `selectedCourse.assignments`, a oceny do każdego z nich znajdowane są metodą `find()` w tablicy `grades`, jeżeli ID pasuje do bieżącego wymaganego zadania (`assignment.id`).

4. Backend

4.1. Rola backendu w aplikacji

Backend to warstwa aplikacji odpowiedzialna za jej logikę i przetwarzanie danych. Jest niewidoczna dla użytkownika, ale to właśnie ona zapewnia poprawne działanie systemu, obsługując wiele kluczowych procesów. Dzięki backendowi możliwe jest m.in. połączenie z bazą danych, w której przechowywane są informacje o użytkownikach, kursach oraz ocenach.

W tej warstwie dane są przetwarzane, zapisywane w bazie lub odczytywane z niej, a następnie udostępniane frontendowi za pośrednictwem API. Backend odpowiada również za integrację z zewnętrznym systemem USOS API, który jest wykorzystywany do logowania użytkowników oraz pobierania ich danych.

4.2. Python Virtual Environment

Aby zapewnić projektowi pełną niezależność i separację od globalnej instalacji Pythona [11], stosuje się wirtualne środowisko Pythona (*Virtual Environment*) [23]. Dzięki temu tworzony projekt może zawierać własny zestaw pakietów i bibliotek w wybranych wersjach, które mogą różnić się od pakietów używanych w innych środowiskach.

Korzystanie z wirtualnego środowiska zapewnia izolację projektu od reszty systemu, co pozwala uniknąć konfliktów wersji oraz problemów wynikających z różnic w zależnościach między projektami. Dzięki temu można bezpiecznie rozwijać oprogramowanie, nie wpływając negatywnie na inne aplikacje korzystające z Pythona. Dodatkowo minimalizuje to ryzyko przypadkowych zmian w globalnych pakietach, które mogłyby wpłynąć na działanie innych programów.

Zastosowanie wirtualnego środowiska pozwala także zabezpieczyć się przed ewentualnymi błędami i uszkodzeniami globalnej instalacji Pythona, które mogłyby wpłynąć na cały system operacyjny.

Cały backend projektu działa w wirtualnym środowisku Pythona, którego wersja to **Python 3.13.1**.

4.3. Architektura backendu – Django i Django Rest Framework

Django [3] to jeden z najpopularniejszych frameworków webowych w języku Python, który pozwala na szybkie tworzenie bezpiecznych i skalowalnych aplikacji internetowych. Jednak w przypadku nowoczesnych aplikacji frontendowych opartych na JavaScript, takich jak React – jak ma to miejsce w naszym projekcie – konieczne jest zapewnienie komunikacji za pomocą REST API [4]. W tym celu stosuje się rozszerzenie Django, czyli Django Rest Framework (DRF) [24].

W aplikacjach jednostronicowych (SPA – Single Page Application) [25], takich jak nasz system oceniania studentów, kluczowe jest dostarczenie API obsługującego zapytania HTTP oraz realizującego standardowe metody: GET, POST, PUT, DELETE. Ponadto API powinno zwracać dane w formacie JSON, co ułatwia ich przetwarzanie po stronie klienta.

W niniejszym projekcie Django pełni rolę backendu API, a także oferuje panel administracyjny w Django Admin. Django Rest Framework (DRF) jest wykorzystywane do obsługi zapytań od frontendowej aplikacji React.

Projekt backendu został podzielony na kilka aplikacji wewnętrznych, z których każda odpowiada za określoną funkcjonalność. Poniżej przedstawiono uproszczoną strukturę katalogów i plików w ramach projektu Django:

4.3.1. Struktura projektu Django

```
backend/
|-- grades/                # Aplikacja logiki
|   |-- tests/             # Testy jednostkowe
|   |-- admin.py           # Rejestracja modeli w panelu admin
|   |-- models.py          # Definicje klas modeli Django
|   |-- serializers.py     # Konwersja modelu -> JSON
|   |-- urls.py            # Routing
|   |-- views.py           # Widoki API
|   |-- migrations/        # Migracje bazy danych
|
|-- oauth/                 # Aplikacja autoryzacji (OAuth)
|   |-- services/
|       |-- usos.py        # Obsługa integracji z USOS API
|   |-- utils/
|       |-- responses.py   # Unifikacja odpowiedzi JSON
|       |-- users.py       # Przetwarzanie danych użytkownika
```

```

|   |-- views/
|       |-- oauth.py      # Obsługa protokołu OAuth
|   |-- exceptions.py     # Obsługa wyjątków
|   |-- urls.py           # Routing autoryzacji
|
|-- student_grades/      # Główna aplikacja projektu
|   |-- settings.py      # Główne ustawienia Django
|   |-- urls.py          # Główny routing aplikacji
|
|-- templates/
|   |-- index.html       # Plik statyczny frontendu HTML
|
|-- static/              # Pliki statyczne (CSS, JS, obrazy)
|-- db.sqlite3           # Plik bazy danych SQLite (dev)
|-- manage.py            # Narzędzie zarządzania projektem
|-- requirements.txt     # Lista zależności projektu
|-- build.sh             # Skrypt wdrożeniowy
|-- .env.development     # Zmienne środowiskowe (dev)
|-- .env.production      # Zmienne środowiskowe (prod)

```

4.4. Omówienie bazy danych

Baza danych jest jednym z kluczowych elementów każdego backendu, ponieważ to w niej przechowywane są wszystkie dane niezbędne do prawidłowego funkcjonowania aplikacji. Dane te są uporządkowane w strukturę podzieloną na tabele, z których każda odpowiada za określony zestaw informacji. Ważnym aspektem projektowania bazy danych jest zadbanie o to, aby jej struktura była na tyle elastyczna, by umożliwiała łatwe dodawanie, modyfikowanie i usuwanie danych, jednocześnie zapewniając ich spójność i integralność.

Dobrze zaprojektowana baza danych powinna być również zoptymalizowana pod kątem wydajności, aby operacje takie jak odczyt, zapis czy wyszukiwanie danych mogły być realizowane szybko i bez zbędnych opóźnień. W tym celu należy zwrócić uwagę na organizację tabel i relacji między nimi, co pozwoli na efektywne zarządzanie danymi i zapewni, że aplikacja będzie działać niezawodnie nawet przy dużym obciążeniu [26].

4.4.1. Struktura bazy danych

W aplikacji zastosowano relacyjną bazę danych, w której dane są organizowane w tabele – encje, a każda z nich składa się z wierszy i kolumn. Wiersze reprezentują rekordy z unikalnymi identyfikatorami, natomiast kolumny zawierają atrybuty – wartości przechowywane dla danego rekordu. Dzięki temu baza danych charakteryzuje się czytelnością i uporządkowaną strukturą.

Baza danych aplikacji składa się z siedmiu encji: Użytkownik (User), Student, Nauczyciel (Teacher), Kurs (Course), Rejestracja (Enrollment), Zadanie (Assignment) i Ocena (Grade).

4.4.2. Struktura encji użytkownika

Model **Użytkownik (User)** pełni rolę nadrzędnej encji względem modeli **Student** i **Nauczyciel (Teacher)**, które przechowują jedynie dodatkowe informacje charakterystyczne dla danego typu użytkownika.

- **Encja Student** zawiera numer studenta.
- **Encja Nauczyciel (Teacher)** jest obecnie pusta, ponieważ wszystkie wspólne dane użytkowników (np. imię, nazwisko, e-mail) są przechowywane w nadrzędnej tabeli **Użytkownik (User)**, jednak jej istnienie jest **świadomym rozwiązaniem projektowym**, które umożliwia przyszłą rozbudowę systemu. W przyszłości tabela **Nauczyciel (Teacher)** może zostać rozszerzona o atrybuty specyficzne dla nauczycieli, takie jak **numer biura, godziny konsultacji czy tytuł naukowy**.

Dzięki takiemu podejściu struktura bazy danych pozostaje **elastyczna i łatwa do rozszerzenia**, bez konieczności wprowadzania fundamentalnych zmian w architekturze.

4.4.3. Relacje między encjami

- **Encja Kurs (Course)** reprezentuje zajęcia prowadzone przez nauczycieli. Przechowuje informacje takie jak nazwa kursu, opis oraz identyfikator prowadzącego.
- **Encja Rejestracja (Enrollment)** odpowiada za zapis studenta na kurs i pełni funkcję **tabeli pośredniczącej** między **Studentem** a **Kursem**.
- **Encja Zadanie (Assignment)** reprezentuje zadania i projekty przypisane do kursu, które podlegają ocenie.

- **Encja Ocena (Grade)** przechowuje **oceny przypisane do konkretnych zadań (Assignment)** realizowanych przez studentów.

Poniżej znajduje się tabela przedstawiająca atrybuty poszczególnych encji wraz z ich typami danych.

User	
Kolumna	Typ
id (PK)	AutoField
email (U)	EmailField
first_name	CharField
last_name	CharField
role	Enum(Student, Teacher, Unknown)

Tabela 1: Struktura tabeli User

Student	
Kolumna	Typ
id (PK)	AutoField
user_id (FK)	ForeignKey(User, on_delete=CASCADE)
student_number (U)	CharField

Tabela 2: Struktura tabeli Student

Teacher	
Kolumna	Typ
id (PK)	AutoField
user_id (FK)	ForeignKey(User, on_delete=CASCADE)

Tabela 3: Struktura tabeli Teacher

Course	
Kolumna	Typ
id (PK)	AutoField
name	CharField
description	TextField
teacher_id (FK)	ForeignKey(Teacher, on_delete=SET_NULL)

Tabela 4: Struktura tabeli Course

Enrollment	
Kolumna	Typ
id (PK)	AutoField
student_id (FK)	ForeignKey(Student, on_delete=CASCADE)
course_id (FK)	ForeignKey(Course, on_delete=CASCADE)

Tabela 5: Struktura tabeli Enrollment

Assignment	
Kolumna	Typ
id (PK)	AutoField
name (IDX)	CharField
description	TextField
weight	DecimalField
is_mandatory	BooleanField
course_id (FK)	ForeignKey(Course, on_delete=CASCADE)

Tabela 6: Struktura tabeli Assignment

Grade	
Kolumna	Typ
id (PK)	AutoField
enrollment_id (FK)	ForeignKey(Enrollment, on_delete=CASCADE)
assignment_id (FK)	ForeignKey(Assignment, on_delete=CASCADE)
score	DecimalField
date_assigned	DateField

Tabela 7: Struktura tabeli Grade

Struktura bazy danych opiera się na kilku kluczowych relacjach między encjami, które zapewniają spójność i integralność danych. Poniżej przedstawiono szczegółowy opis każdej relacji.

4.4.4. Relacja między Użytkownik (User), Student i Nauczyciel (Teacher) (1:1)

Każdy użytkownik (**User**) może być albo studentem (**Student**), albo nauczycielem (**Teacher**). Jest to realizowane za pomocą relacji **jeden do jednego (1:1)**, co zapobiega sytuacji, w której użytkownik pełni obie role jednocześnie.

4.4.5. Relacja między Nauczyciel (Teacher) i Kurs (Course) (1:M)

Każdy nauczyciel (**Teacher**) może prowadzić wiele kursów (**Course**), ale każdy kurs ma tylko jednego prowadzącego. Jest to przykład relacji **jeden do wielu (1:M)**.

4.4.6. Relacja między Student (Student) i Kurs (Course) poprzez Rejestrację (Enrollment) (M:N)

Studenci mogą zapisywać się na różne kursy, a każdy kurs może mieć wielu studentów. Ponieważ w bazach relacyjnych bezpośrednia relacja **wiele do wielu (M:N)** nie jest możliwa, zastosowano tabelę pośrednią **Enrollment**, która łączy studentów z kursami.

Każdy wpis w **Enrollment** reprezentuje jedno przypisanie studenta do kursu oraz wymusza **unikalność pary** (`student_id`, `course_id`), co zapobiega wielokrotnym zapisom tego samego studenta na ten sam kurs.

4.4.7. Relacja między Zadanie (Assignment) i Kurs (Course) (1:M)

Każde zadanie (**Assignment**) jest przypisane do jednego kursu (**Course**), ale każdy kurs może mieć wiele zadań. Jest to relacja **jeden do wielu (1:M)**.

4.4.8. Relacja między Rejestracja (Enrollment), Zadanie (Assignment) i Ocena (Grade) (1:M)

Oceny (**Grade**) są przypisywane do zadań (**Assignment**), ale ocena powinna być związana z konkretnym studentem zapisanym na kurs. Dlatego tabela **Grade** nie łączy się bezpośrednio z **Student**, lecz z **Enrollment**, co zapewnia, że oceny mogą być przypisane tylko do studentów, którzy faktycznie są zapisani na dany kurs. Jest to przykład relacji **jeden do wielu (1:M)**.

4.4.9. Podsumowanie relacji między encjami

Poniższa tabela przedstawia podsumowanie wszystkich relacji w bazie danych:

Encja 1	Encja 2	Typ relacji
User	Student	Jeden-do-jednego (1:1)
User	Teacher	Jeden-do-jednego (1:1)
Teacher	Course	Jeden-do-wielu (1:M)
Student	Course (via Enrollment)	Wiele-do-wielu (M:N)
Enrollment	Grade	Jeden-do-wielu (1:M)
Assignment	Course	Wiele-do-jednego (M:1)
Grade	Assignment	Wiele-do-jednego (M:1)

Tabela 8: Podsumowanie relacji między encjami

4.5. Modele

Jednym z kluczowych elementów każdej aplikacji Django są modele [27], które definiują strukturę i sposób przechowywania danych w bazie. Django opiera się na podejściu ORM (*Object-Relational Mapping*), co oznacza, że zamiast bezpośredniego tworzenia tabel SQL, programista definiuje modele jako klasy Pythona, a Django automatycznie przekształca je w odpowiednie tabele bazodanowe. Dzięki temu dane są reprezentowane jako obiekty, a operacje na nich mogą być wykonywane za pomocą kodu Pythona.

4.5.1. Modele Django a encje bazy danych

Wcześniej przedstawiono encje bazy danych oraz ich wzajemne relacje. W Django każda encja jest odwzorowana jako model, który dziedziczy po klasie `models.Model`. Dzięki temu Django ORM automatycznie mapuje modele na tabele w bazie danych, co pozwala na operacje na danych za pomocą kodu Pythona, zamiast bezpośrednich zapytań SQL.

W projekcie modele Django odpowiadają wcześniej omówionym encjom:

- **Modele użytkowników:**

- **User** – reprezentuje użytkownika systemu.
- **Student** – rozszerzenie modelu **User**, przechowujące dodatkowe informacje o studentach.
- **Teacher** – rozszerzenie modelu **User**, przechowujące informacje o nauczycielach.

- **Modele kursów i zapisów:**

- **Course** – odpowiada kursom prowadzonym w systemie.
- **Enrollment** – model pośredniczący między **Student** a **Course**, obsługujący zapisy na kursy.
- **Modele zadań i ocen:**
 - **Assignment** – przechowuje informacje o zadaniach w kursach.
 - **Grade** – przechowuje oceny studentów.

4.5.2. Przykładowa implementacja modelu

Poniżej przedstawiono przykład modelu **Student**, który przechowuje dane studentów:

```
class Student(models.Model):
    """ ... """

    id = models.AutoField(primary_key=True)
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name="student")
    student_number = models.CharField(max_length=10, unique=True, db_index=True)

    def __str__(self):
        return f"{self.user.first_name} {self.user.last_name} - {self.student_number}"
```

Rysunek 16: Klasa Student

Model **Student** w Django reprezentuje pojedynczego studenta w systemie. Jest on bezpośrednio związany z modelem **User** poprzez relację jeden-do-jednego za pomocą **OneToOneField**, zapewniając tym samym, że każdy student odpowiada unikatowemu kontu użytkownika. Model ten zawiera kluczowe i unikatowe dla studenta dane, takie jak numer studenta (**student_number**).

4.5.3. Identyfikator główny w modelach

```
id = models.AutoField(primary_key=True)
```

Pole **id** jest automatycznie inkrementowanym identyfikatorem, a **primary_key=True** oznacza, że jest to klucz główny (ang. *primary key*) [28] w bazie danych, zapewniający unikalną identyfikację każdego obiektu. Django domyślnie generuje to pole, nawet jeśli nie zostanie jawnie zdefiniowane.

4.5.4. Relacja z modelem użytkownika

```
user = models.OneToOneField(User, on_delete=models.CASCADE,  
                             related_name="student")
```

To pole tworzy relację jeden-do-jednego pomiędzy modelem `Student` a modelem `User`. Każdy student musi być przypisany do jednego użytkownika. Parametr `related_name="student"` umożliwia łatwy dostęp do obiektu studenta poprzez użytkownika, np. `user.student`.

4.5.5. Znaczenie `on_delete=models.CASCADE`

Parametr `on_delete=models.CASCADE` w polu `OneToOneField` definiuje, co powinno się stać, gdy powiązany obiekt użytkownika (`User`) zostanie usunięty. W tym przypadku, jeśli użytkownik zostanie usunięty, Django automatycznie usunie również przypisanego do niego studenta. Mechanizm ten zapewnia spójność danych, zapobiegając pozostawianiu rekordów bez powiązanych użytkowników.

Inne możliwe wartości parametru `on_delete`:

- `models.SET_NULL` – ustawia wartość `NULL` w miejscu usuniętego użytkownika.
- `models.PROTECT` – blokuje usunięcie użytkownika, jeśli istnieje powiązany student.
- `models.DO_NOTHING` – nie podejmuje żadnych działań (może prowadzić do błędów integralności bazy danych).

4.5.6. Modele wykorzystujące `on_delete=models.CASCADE`

W projekcie zastosowano usuwanie kaskadowe w następujących modelach:

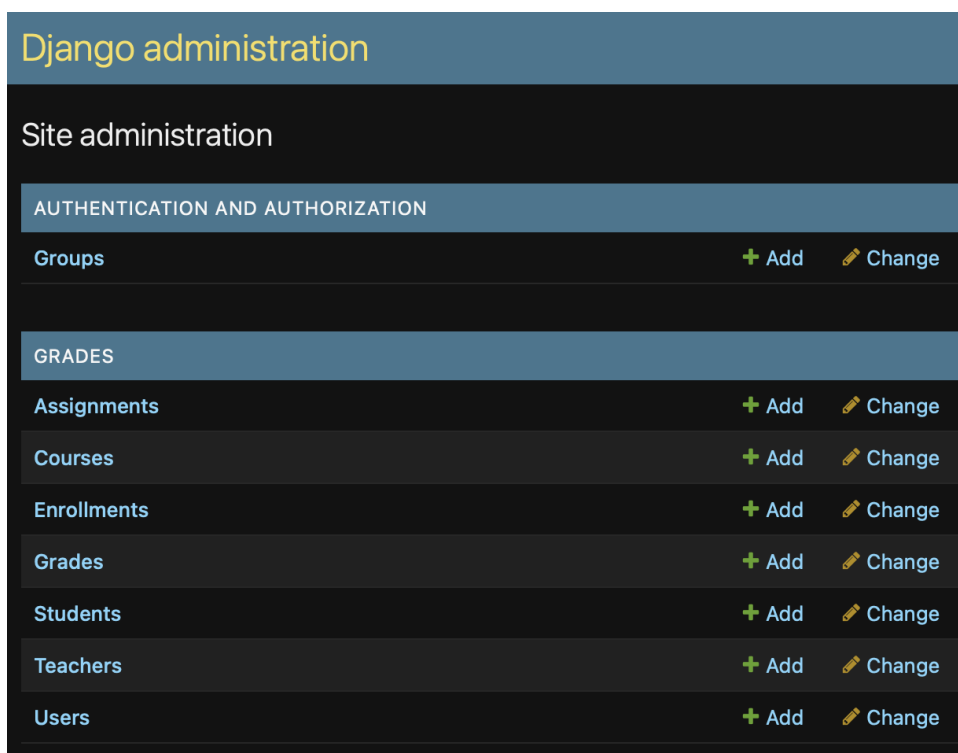
- **Student** – usunięcie `User` powoduje usunięcie powiązanego studenta.
- **Teacher** – usunięcie `User` powoduje usunięcie powiązanego nauczyciela.
- **Enrollment** – usunięcie `Student` lub `Course` powoduje usunięcie wszystkich związanych rekordów rejestracji.
- **Grade** – usunięcie `Enrollment` lub `Assignment` powoduje usunięcie przypisanych ocen.
- **Assignment** – usunięcie `Course` powoduje usunięcie wszystkich jego zadań.

4.5.7. Metoda `__str__`

Metoda `__str__` zwraca czytelną reprezentację obiektu, dzięki czemu Django wyświetla go w panelu administracyjnym i ułatwia debugowanie.

4.6. Django Admin

Jedną z cech frameworka Django, wyróżniającą go wśród innych oraz ułatwiającą pracę deweloperów, jest automatyczny interfejs administracyjny [29] dostarczany w trakcie tworzenia nowej aplikacji. Dzięki niemu możliwe jest odczytywanie metadanych z wcześniej stworzonych modeli, co pozwala autoryzowanym użytkownikom na wygodne zarządzanie danymi aplikacji. Głównym celem tej funkcji jest szybka organizacja i wizualizacja danych, jednak nie jest ona przeznaczona do dalszej rozbudowy ani jako fundament do budowania całego interfejsu użytkownika (frontendu).



Rysunek 17: Wygląd panelu administracyjnego

Django Admin oferuje wiele możliwości personalizacji. Jest on domyślnie dostępny pod adresem `/admin/`. Strona administracyjna jest chroniona hasłem, dlatego aby uzyskać do niej dostęp, należy utworzyć konto superużytkownika poprzez wykonanie następującej komendy:

```
python manage.py createsuperuser
```

Następnie należy postępować zgodnie z wyświetlanymi instrukcjami.

```
.venvpiotrglajcar@MacBook-Air-Piotr backend % python manage.py createsuperuser
Email: admin@example.com
First name: Jan
Last name: Kowalski
Password:
Password (again): █
```

Rysunek 18: Proces tworzenia konta superużytkownika

W ten sposób można uzyskać dostęp administracyjny do zasobów aplikacji.

4.6.1. Rejestracja modeli w Django Admin

Użytkownik może zdecydować, którymi modelami aplikacji Django będzie mógł zarządzać w panelu administracyjnym. Każdy z tych modeli należy zarejestrować w pliku `admin.py` poprzez dekorator `@admin.register(Model)`, gdzie w miejsce `Model` należy wprowadzić nazwę rejestrowanego modelu.

Dla przykładu zostanie omówiony tutaj model użytkownika aplikacji. Przechowuje on dane o wszystkich osobach korzystających z aplikacji, w tym ich role, uprawnienia oraz szczegółowe informacje, takie jak data dołączenia czy ostatnie logowanie.

4.6.2. Konfiguracja widoków w Django Admin

Django pozwala na dostosowanie wyświetlanych informacji w widoku panelu administracyjnego. W zależności od potrzeb i wymagań aplikacji można dostosować sposób wyświetlania pobieranych z bazy danych rekordów, a także spersonalizować panel edycji oraz formularz tworzenia nowych rekordów.

Do tego celu służą następujące opcje:

Lista użytkowników Opcja `list_display` określa, które pola mają być widoczne w głównym widoku administracyjnym użytkowników. Kolejność pól odpowiada układowi w panelu administratora.

Widok szczegółowy użytkownika Opcja `fieldsets` odpowiada za szczegółowy widok użytkownika, umożliwiając również edycję jego danych.

Formularz dodawania użytkownika Opcja `add_fieldsets` określa, jakie pola mają być ustawione przez administratora podczas dodawania nowego użytkownika.

```

8  @admin.register(User)
9  class UserAdmin(BaseUserAdmin):
10 >  """ ...
13
14  list_display = ("email", "first_name", "last_name", "role", "is_staff", "is_active")
15  list_filter = ("role", "is_staff", "is_superuser", "is_active")
16
17  fieldsets = (
18      (None, {"fields": ("email", "password")}),
19      ("Personal info", {"fields": ("first_name", "last_name")}),
20      (
21          "Roles and Permissions",
22          {"fields": ("role", "is_staff", "is_superuser", "is_active")},
23      ),
24      ("Important dates", {"fields": ("last_login", "date_joined")}),
25  )
26
27  add_fieldsets = (
28      (
29          None,
30          {
31              "classes": ("wide",),
32              "fields": (
33                  "email",
34                  "first_name",
35                  "last_name",
36                  "password1",
37                  "password2",
38                  "role",
39                  "is_staff",
40                  "is_active",
41              ),
42          },
43      ),
44  )

```

Rysunek 19: Klasa użytkownika w rejestracji administratora

EMAIL	FIRST NAME	LAST NAME	2 ▲ ROLE	3 ▼ STAFF STATUS	1 ▲ ACTIVE
pg300857@student.polsl.pl	Piotr	Głajcar	Student	○	●

Rysunek 20: Widok ogólny – lista użytkowników odpowiada polu list_display

Change user

Piotr Glajcar

Email:

pg300857@student.polsl.pl

Password:

No password set.

Set password

Raw passwords are not stored, so there is no way to see the user's password.

Personal info

First name:

Piotr

Last name:

Glajcar

Roles and Permissions

Role:

Student

☐ Staff status

Designates whether the user can log into this admin site.

☐ Superuser status

Designates that this user has all permissions without explicitly assigning them.

☒ Active

Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

Important dates

Last login:

Date:

2025-02-16

Today |

Time:

18:17:47

Now |

Note: You are 1 hour ahead of server time.

Date joined:

Date:

2025-02-13

Today |

Time:

18:14:59

Now |

Note: You are 1 hour ahead of server time.

SAVE

Save and add another

Save and continue editing

Rysunek 21: Widok szczegółowy użytkownika

Add user

First, enter a username and password. Then, you'll be able to edit more user options.

Email:

First name:

Last name:

Password:

Your password can't be too similar to your other personal information.
Your password must contain at least 8 characters.
Your password can't be a commonly used password.
Your password can't be entirely numeric.

Password confirmation:

Enter the same password as before, for verification.

Role:

☐ Staff status
Designates whether the user can log into this admin site.

☒ Active
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

Rysunek 22: Widok dodawania nowego użytkownika

4.7. OAuth 1.0a – protokół autoryzacji

Jednym z największych wyzwań tworzonej aplikacji była kwestia autoryzowania użytkowników, a co za tym idzie, bezpieczeństwa aplikacji oraz – co ważniejsze – ochrony danych użytkowników i bezpiecznego przechowywania ich informacji logowania. Równocześnie zauważono potencjał uczelnianego systemu USOS oraz jego bogato rozbudowanego API [31], które umożliwia deweloperom dostęp do akademickiej bazy danych.

Zatem zdecydowano się, aby główną rolę w procesie autoryzacji pełnił zewnętrzny system USOS. Użytkownik aplikacji, po kliknięciu przycisku logowania, zostaje przekierowany na stronę logowania systemu USOS. Po pomyślnej weryfikacji użytkownika jest on przekierowywany z powrotem do aplikacji, natomiast tym samym aplikacja zyskuje pozwolenie na pobranie niezbędnych danych użytkownika.

USOS API wykorzystuje protokół OAuth 1.0a [30] do autoryzacji. Najpierw przybliżę czytelnikowi, czym jest OAuth 1.0a, a następnie przedstawię jego działanie na przykładzie naszej aplikacji. Kluczem do zrozumienia, jak działa OAuth, jest poznanie przepływu autoryzacji. Jest to proces, przez który przechodzi klient (aplikacja), aby uzyskać dostęp do zasobów chronionych na serwerze. Opiera się on na trzech głównych etapach:

1. **Uzyskanie tymczasowego tokenu żądania** – Klient otrzymuje tymczasowy token autoryzacyjny z serwera.
2. **Autoryzacja tokenu żądania** – Użytkownik autoryzuje token żądania, umożliwiając dostęp do swojego konta.
3. **Wymiana tokenu** – Klient wymienia krótkotrwały token żądania na token dostępu o dłuższej żywotności.

4.7.1. Uzyskanie tymczasowego tokenu żądania

Pierwszym krokiem w procesie autoryzacji jest uzyskanie tymczasowego tokenu żądania (ang. *request token*). Token ten jest krótkotrwały (zwykle ważny do 24 godzin) i służy wyłącznie do rozpoczęcia procesu autoryzacji. Nie zapewnia dostępu do danych użytkownika i nie może być użyty do niczego innego poza autoryzacją.

Tymczasowy token żądania uzyskuje się poprzez wysłanie żądania do odpowiedniego endpointu serwera autoryzacyjnego. Klient inicjuje to zapytanie, podając następujące parametry:

- **klucz konsumenta** (*consumer key*) – unikalny identyfikator aplikacji,

- **adres zwrotny** (*callback URL*) – adres, na który użytkownik zostanie przekierowany po pomyślnej autoryzacji,
- **znacznik zapytania** (*nonce*) – unikalna wartość zabezpieczająca przed atakami powtórzeniowymi (replay attacks).

Po otrzymaniu żądania serwer autoryzacyjny weryfikuje poprawność klucza konsumenta i znacznika zapytania, aby upewnić się, że klient jest autoryzowany do korzystania z usługi. Następnie sprawdza poprawność przekazanego adresu zwrotnego. Jeśli wszystkie warunki są spełnione, serwer generuje nowy zestaw danych uwierzytelniających:

- **token żądania** (*request token*),
- **tajny klucz tokenu** (*token secret*).

Te dane są zwracane w odpowiedzi HTTP i będą używane w dalszych etapach autoryzacji oraz wymiany tokenów.

4.7.2. Autoryzacja

Kolejnym krokiem w procesie OAuth 1.0a jest autoryzacja użytkownika. Jest to etap wymagający interakcji użytkownika aplikacji, który może już być znany z innych systemów autoryzacyjnych. Jest to proces logowania się użytkownika aplikacji.

Klient korzysta z adresu URL autoryzacji dostarczonego przez serwer, do którego dołącza tymczasowy token żądania (*request token*) jako parametr zapytania. Następnie klient przekierowuje użytkownika do tego adresu. W praktyce odbywa się to poprzez przekierowanie użytkownika w jego przeglądarce lub otwarcie nowego okna autoryzacji.

Po przejściu na stronę autoryzacji użytkownik loguje się (jeśli nie jest jeszcze zalogowany) i decyduje, czy chce autoryzować dostęp dla klienta. Może również anulować proces autoryzacji, jeśli nie chce nawiązywać połączenia z klientem.

Jeżeli użytkownik wyrazi zgodę, serwer oznacza token żądania jako autoryzowany i przekierowuje użytkownika z powrotem do wcześniej podanego adresu zwrotnego (*callback URL*). Adres ten zawiera dwa dodatkowe parametry zapytania:

- **token** – ten sam tymczasowy token żądania, który został wysłany na początku procesu autoryzacji,
- **verifier** – jednorazowy kod autoryzacyjny (ang. *oauth verifier*), który musi zostać przesłany w kolejnym kroku w celu uzyskania tokenu dostępu.

4.7.3. Wymiana tokenów

Finalnym krokiem w procesie autoryzacji jest wymiana tymczasowego tokenu żądania (*request token*) na token dostępu (*access token*) o dłuższej żywotności oraz unieważnienie tymczasowego tokenu. Tymczasowe dane uwierzytelniające (*credentials*) są konwertowane na trwałe poprzez wysłanie żądania do endpointu obsługującego wymianę tokenów.

Żądanie wymiany tokenu musi być:

- podpisane przy użyciu tymczasowych danych uwierzytelniających (tokenu żądania i tajnego klucza tokenu),
- zawierać kod weryfikacyjny (*oauth verifier*) uzyskany w poprzednim kroku autoryzacji.

Serwer ponownie weryfikuje klucz konsumenta, podpis żądania oraz kod weryfikacyjny, aby upewnić się, że proces przebiega zgodnie z zasadami bezpieczeństwa i zapobiega atakom typu *replay* lub CSRF. Jeśli wszystkie warunki zostaną spełnione, serwer zwraca w odpowiedzi HTTP końcowy zestaw danych uwierzytelniających:

- **token dostępu** (*access token*),
- **tajny klucz tokenu dostępu** (*access token secret*).

4.7.4. Wykorzystanie tokenu dostępu

Po pomyślnej wymianie tokenów klient może autoryzować swoje żądania do API za pomocą tokenu dostępu. Każde żądanie musi być podpisane tokenem i tajnym kluczem tokenu dostępu, co zapewnia integralność i autoryzację żądań.

Przykładowe żądanie:

```
GET /api/user_info HTTP/1.1
Host: usosapi.example.com
Authorization: OAuth oauth_consumer_key="abc123",
oauth_token="xyz789",
oauth_signature="generated_signature"
```

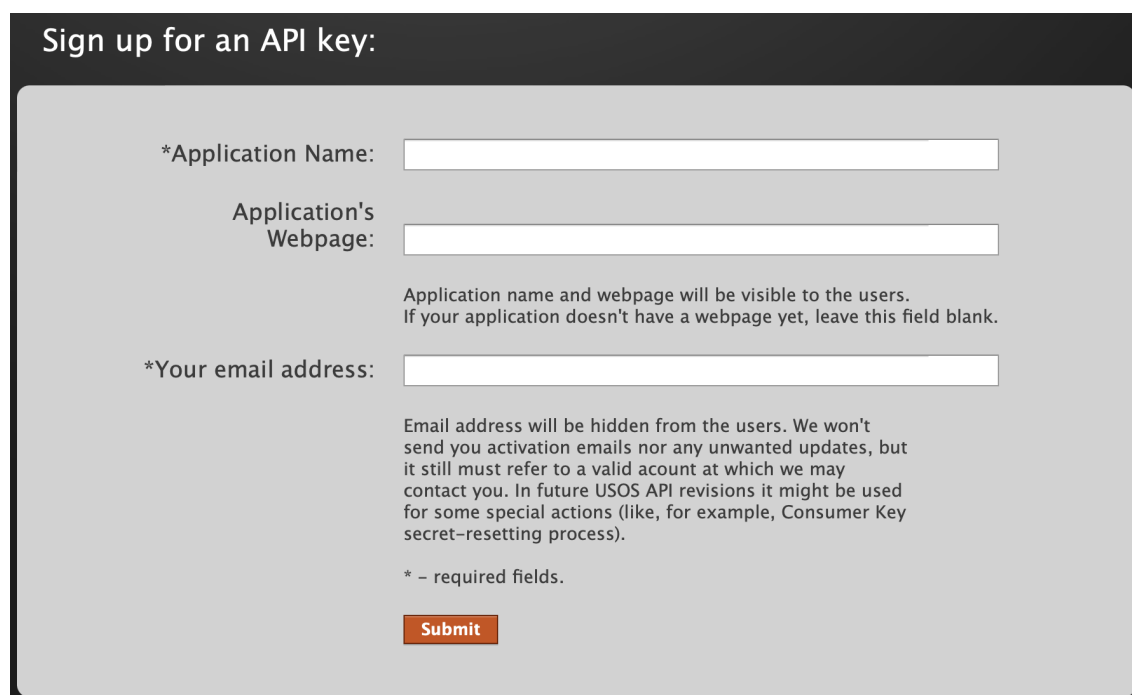
Serwer API weryfikuje podpis i poprawność tokenu, a następnie zwraca zasoby dostępne dla uwierzytelnionego użytkownika.

4.8. Integracja aplikacji z USOS API

Aplikacja wykorzystuje system autoryzacji USOS API oparty na protokole OAuth 1.0a. Cały proces składa się z trzech kluczowych etapów: pobrania tymczasowego tokenu, autoryzacji użytkownika oraz wymiany tokenu żądania na token dostępu.

4.8.1. Uzyskanie klucza konsumenta i consumer secrets

Aby jakakolwiek interakcja z API USOS była możliwa, klient musi zarejestrować aplikację w systemie, gdzie otrzymuje wygenerowane dane. To dzięki nim możliwe jest przejście pierwszego kroku protokołu autoryzacji.



The image shows a web form titled "Sign up for an API key:". It contains three input fields, each preceded by an asterisk indicating it is a required field. The first field is labeled "*Application Name:" and is a single-line text input. The second field is labeled "Application's Webpage:" and is a single-line text input. The third field is labeled "*Your email address:" and is a single-line text input. Below the "Application's Webpage:" field, there is a note: "Application name and webpage will be visible to the users. If your application doesn't have a webpage yet, leave this field blank." Below the "*Your email address:" field, there is a note: "Email address will be hidden from the users. We won't send you activation emails nor any unwanted updates, but it still must refer to a valid account at which we may contact you. In future USOS API revisions it might be used for some special actions (like, for example, Consumer Key secret-resetting process)." At the bottom of the form, there is a small orange button labeled "Submit". A legend at the bottom left states "* - required fields."

Rysunek 23: Formularz rejestracji aplikacji w USOS API

Application Name: **UniGradeFlow**

Consumer Key: **FyjjjQW6MDy4tRGNTQke**

Consumer Secret: **uk7hQQggkpMPPpZUPDhbTLYcvXGDCTfwnrjUxCDM**

Paste these into your application and start using USOS API!
Remember: Keep them safe!

Rysunek 24: Przykładowe klucze otrzymane po zarejestrowaniu aplikacji w USOS API

4.8.2. Dane użytkownika pobierane z USOS API

Nasza aplikacja, jako klient systemu USOS, pobiera informacje o użytkowniku:

- identyfikator użytkownika (ID) w systemie USOS,
- imię,
- nazwisko,
- adres e-mail,
- numer albumu studenta,
- status studenta,
- status pracownika.

W podstawowym zapytaniu o informacje studenta można otrzymać jedynie ID, imię i nazwisko. Aby rozszerzyć te dane o pozostałe informacje, należy do zapytania dołączyć tzw. *scope* – czyli zakres danych, na których pobranie klient prosi użytkownika aplikacji o zgodę.

USOS API oferuje szeroki wybór dostępnych danych, jednak w naszej aplikacji pobieramy jedynie niezbędne informacje, korzystając z następujących *scopes*: **email**, **studies**, **personal**.

- **email** – umożliwia pobranie adresu e-mail użytkownika,
- **studies** – pozwala uzyskać numer albumu studenta,
- **personal** – zawiera informacje o statusie studenta i pracownika.

4.8.3. Proces inicjalizacji OAuth

Wszystko zaczyna się w funkcji `initiate_oauth(request)`, która rozpoczyna proces autoryzacji OAuth z API USOS. Na początku definiowane są *scopes* (zakresy danych):

```
scopes = "email|studies|personal"
```

Następnie, z określonym zakresem, wywoływana jest metoda `fetch_request_token` klasy `UsosAPI`:

```
tokens = usos_api.fetch_request_token(scopes=scopes)
```

Metoda ta zwraca słownik (*dictionary*) z tokenami żądania i przypisuje je do zmiennej `tokens`.

Przed tym, jak zwrócone zostaną tokeny, metoda `fetch_request_token` tworzy zmienną `session`, która przechowuje wymagane przez USOS API parametry niezbędne do uwierzytelnienia:

```
client_key=self.consumer_key ,  
client_secret=self.consumer_secret ,  
resource_owner_key=resource_owner_key ,  
resource_owner_secret=resource_owner_secret ,  
callback_uri=self.callback_url ,  
verifier=verifier ,
```

Następnie tworzony jest endpoint w USOS API, który odpowiada za wydanie tokenu. Do zapytania dodawane są *scopes*, a następnie wywoływana jest metoda `fetch_request_token` z biblioteki `requests_oauthlib`:

```
from requests_oauthlib import OAuth1Session
```

Parametrem tej metody jest wcześniej utworzony endpoint. W odpowiedzi zwracane są tokeny.

Tokeny są przechowywane w sesji użytkownika, co pozwala na ich późniejsze wykorzystanie w kolejnych krokach autoryzacji (np. wymiany na tokeny dostępu):

```
request.session["oauth_token"] = tokens["oauth_token"]  
request.session["oauth_token_secret"]  
    = tokens["oauth_token_secret"]
```

Następnie tworzony jest URL, na który użytkownik zostaje przekierowany w celu zalogowania się do USOS i autoryzowania aplikacji. Token jest przekazywany jako parametr:

```
authorization_url = usos_api.get_authorization_url(
    tokens["oauth_token"])
return redirect(authorization_url)
```

Metoda `get_authorization_url`, oprócz generowania adresu URL, tworzy sesję OAuth przy użyciu przekazanego tokena oraz powiązuje sesję z konkretnym użytkownikiem, który jeszcze nie dokończył autoryzacji:

```
session = self.get_oauth_session(
    resource_owner_key=oauth_token)
return session.authorization_url(
    f"{self.base_url}/services/oauth/authorize")
```

Ostatecznie użytkownik jest przekierowywany do wygenerowanego adresu URL w celu dokończenia procesu autoryzacji.

4.8.4. Proces autoryzacji w funkcji `initiate_oauth`

Następnym krokiem procesu autoryzacji jest wywołanie funkcji `initiate_oauth`, która pobiera token żądania, *verifier* oraz tokeny sesji. Jeśli coś pójdzie nie tak, aplikacja zwróci błąd:

```
oauth_token = request.GET.get("oauth_token")
oauth_verifier = request.GET.get("oauth_verifier")
oauth_token_secret =
    request.session.get("oauth_token_secret")

if not oauth_token or not oauth_verifier or not
    oauth_token_secret:
    return api_response(
        status="error",
        message="Missing OAuth tokens",
        error_code="MISSING_TOKENS",
        status_code=400,
    )
```

Następnie kod obsługuje końcowy etap procesu OAuth w USOS API – wymianę tymczasowych tokenów na tokeny dostępu, pobranie informacji o użytkowniku oraz zalogowanie go w systemie.

```
access_tokens = usos_api.fetch_access_token(
    oauth_token, oauth_token_secret, oauth_verifier
```

)

Wywołanie funkcji `fetch_access_token` powoduje wymianę tymczasowych tokenów oraz *verifier* na trwały token dostępu.

4.8.5. Pobieranie tokena dostępu

Metoda `fetch_access_token` tworzy nową sesję OAuth, która jest uwierzytelniana za pomocą tokenów oraz *verifier* przekazanego jako parametr:

```
def fetch_access_token(self, oauth_token, oauth_token_secret,
    verifier):
    session = self.get_oauth_session(
        resource_owner_key=oauth_token,
        resource_owner_secret=oauth_token_secret,
        verifier=verifier,
    )
```

Następnie wysyłane jest żądanie ostatecznego uzyskania tokena dostępu:

```
response = session.fetch_access_token(
    f"{self.base_url}/services/oauth/access_token"
)
return response
```

Tokeny zwrócone w odpowiedzi mogą być dalej wykorzystywane:

- `access_tokens["oauth_token"]` – finalny token dostępu użytkownika,
- `access_tokens["oauth_token_secret"]` – tajny klucz dostępu.

Tokeny te są zapisywane w sesji użytkownika, dzięki czemu aplikacja może później używać ich do wysyłania autoryzowanych żądań w jego imieniu:

```
request.session["access_token"] =
    access_tokens["oauth_token"]
request.session["access_token_secret"] =
    access_tokens["oauth_token_secret"]
```


4.8.6. Pobieranie informacji o użytkowniku

Następnie wywoływana jest metoda `get_usos_user_data`, która jako parametry przyjmuje tokeny dostępu oraz określa pola, które aplikacja chce uzyskać w ramach danych użytkownika:

```
user_info = usos_api.get_usos_user_data(  
    access_token=access_tokens["oauth_token"],  
    access_token_secret=access_tokens["oauth_token_secret"],  
    fields="id|first_name|last_name|email|student_number  
           |student_status|staff_status",  
)
```

W metodzie tworzona jest sesja OAuth, w której użytkownik jest uwierzytelniony. Parametry `resource_owner_key` i `resource_owner_secret` pozwalają na autoryzowany dostęp do jego danych:

```
session = self.get_oauth_session(  
    resource_owner_key=access_token,  
    resource_owner_secret=access_token_secret  
)
```

Następnie odczytywane są parametry, o które aplikacja chce zapytać USOS API. Parametry te zostają dodane do żądania, określając, które konkretne dane użytkownika mają zostać zwrócone.

Adres żądania to endpoint w API USOS, który zwraca dane użytkownika, natomiast parametry wyznaczają zakres zwracanych informacji:

```
params = {"fields": fields} if fields else {}  
response = session.get(f"{self.base_url}/services/users/user",  
    params=params)  
response.raise_for_status()  
return response.json()
```

4.8.7. Przetwarzanie danych użytkownika

Po pobraniu danych użytkownika następuje ich przetworzenie w metodzie `process_usos_user`. Dane te są przekazywane jako parametr, z którego odczytywane są konkretne pola i przypisywane do odpowiednich zmiennych, tak aby można było z nich korzystać w dalszej części programu:

```
def process_usos_user(user_info):
```

```

email = user_info.get("email")
first_name = user_info.get("first_name", "")
last_name = user_info.get("last_name", "")
student_number = user_info.get("student_number")
student_status = user_info.get("student_status")
staff_status = user_info.get("staff_status")

```

Następnie wyszukiwany jest użytkownik w bazie danych na podstawie adresu e-mail.

- Jeśli użytkownik już istnieje, nie jest tworzony nowy rekord.
- Jeśli użytkownik nie istnieje, tworzony jest nowy rekord, a jego domyślne wartości to imię, nazwisko oraz nieokreślona rola:

```

user, created = User.objects.get_or_create(
    email=email,
    defaults={
        "first_name": first_name,
        "last_name": last_name,
        "role": "unknown",
    },
)

```

4.8.8. Określanie roli użytkownika

Wraz z danymi użytkownika pobierane są również wartości pól `student_status` i `staff_status`, które mogą przyjmować wartości:

- **Null** – aplikacja nie ma dostępu do danych użytkownika.
- **0** – użytkownik nie jest i nigdy nie był studentem żadnego programu na uczelni / nigdy nie był pracownikiem (lub był, ale już nie jest).
- **1** – użytkownik jest nieaktywnym studentem (kiedyś studiował, ale obecnie nie studiuje) / jest pracownikiem, ale nie nauczycielem akademickim.
- **2** – użytkownik jest aktywnym studentem / jest aktywnym nauczycielem akademickim.

Na podstawie wartości `staff_status` i `student_status` użytkownikowi przypisywana jest odpowiednia rola:

```

if created or user.role == "unknown":
    if staff_status == 2:
        user.role = "teacher"
    elif student_status == 2:
        user.role = "student"
    else:
        user.role = "unknown"
user.save()

```

4.8.9. Tworzenie obiektów studenta i nauczyciela

Jeśli użytkownik ma przypisaną rolę studenta, tworzony jest obiekt klasy `Student`, a do jego danych dodawany jest numer albumu:

```

if user.role == "student":
    Student.objects.get_or_create(
        user=user,
        defaults={"student_number": student_number},
    )

```

Analogicznie, jeśli użytkownik ma przypisaną rolę nauczyciela, tworzony jest obiekt klasy `Teacher`:

```

elif user.role == "teacher":
    Teacher.objects.get_or_create(user=user)

```

4.8.10. Logowanie użytkownika i przekierowanie

Ostatnim krokiem jest zalogowanie użytkownika przy użyciu wewnętrznej funkcji Django, która uwierzytelnia użytkownika i zapisuje go w sesji. Dzięki temu użytkownik nie musi ponownie się logować przy każdej interakcji z aplikacją:

```
login(request, user)
```

Po pomyślnym zalogowaniu aplikacja przekierowuje użytkownika na adres należący do frontendu `/redirect`, gdzie następują dalsze działania:

```
return redirect("/redirect")
```

4.9. Pliki w projekcie backendu

4.9.1. manage.py

Plik `manage.py` to kluczowe narzędzie do zarządzania projektem Django. Znajduje się w głównym katalogu projektu i pozwala na wykonywanie poleceń administracyjnych, takich jak migracje bazy danych, uruchamianie serwera deweloperskiego czy testowanie aplikacji.

Najważniejsze polecenia:

- **Uruchomienie serwera deweloperskiego:** `python manage.py runserver`
Uruchamia wbudowany serwer HTTP Django (domyślnie na `127.0.0.1:8000`).
- **Zarządzanie migracjami:** `python manage.py makemigrations` – generuje pliki migracji na podstawie zmian w modelach. `python manage.py migrate` – stosuje migracje do bazy danych.
- **Tworzenie superużytkownika:** `python manage.py createsuperuser` Pozwala na utworzenie konta administratora do panelu Django Admin.
- **Uruchamianie testów jednostkowych:** `python manage.py test`
- **Czyszczenie bazy danych:** `python manage.py flush` Usuwa wszystkie dane z bazy, zachowując jej strukturę.

4.9.2. serializers.py

Serializatory w Django REST Framework (*DRF*) odpowiadają za konwersję danych między wewnętrzną logiką aplikacji a formatem JSON, który może być zwracany jako odpowiedź API. Pozwalają one zarówno na **serializację** (przekształcanie obiektów Pythona na JSON), jak i **deserializację** (odczytywanie JSON i zamianę go na obiekty Django/Pythona).

Serializatory określają również, jakie dane zostaną zwrócone w odpowiedzi API oraz umożliwiają **walidację danych wejściowych**, co pomaga w kontrolowaniu poprawności przesyłanych informacji.

Przykład prostego serializatora:

```
from rest_framework import serializers
from .models import Course

class CourseSerializer(serializers.ModelSerializer):
    class Meta:
```

```
model = Course
fields = ['id', 'name', 'description', 'teacher_id']
```

Serializator ten automatycznie pobierze dane z modelu `Course` i zwróci je w formacie JSON, a także pozwoli na ich zapis po przesłaniu poprawnych danych do API.

4.9.3. responses.py

Jest to niestandardowa funkcja (*custom function*), która porządkuje i ujednolica format zwracanych przez API danych. Dzięki temu odpowiedzi API mają spójny wygląd, co ułatwia ich odczytanie oraz obsługę na frontendzie.

Każda odpowiedź zawiera trzy kluczowe pola:

- **status** – określa, czy akcja zakończyła się powodzeniem ("success") czy błędem ("error").
- **message** – krótka informacja tekstowa rozszerzająca komunikat **status**, np. szczegółowy błąd lub potwierdzenie poprawnego wykonania operacji.
- **data** – zawiera właściwe dane zwrócone w odpowiedzi na zapytanie.

Dzięki takiemu podejściu API zwraca odpowiedzi w zunifikowanym formacie, co ułatwia ich obsługę po stronie frontendowej i pozwala na spójne zarządzanie błędami oraz sukcesami.

Przykładowa odpowiedź zwracana przez API:

```
{
  "status": "success",
  "message": "Courses retrieved successfully.",
  "data": [
    {"id": 1, "name": "Mathematics"},
    {"id": 2, "name": "Physics"}
  ]
}
```

4.9.4. views.py

Pliki `views.py` odpowiada za obsługę logiki API w Django REST Framework (*DRF*). Widoki określają, jakie operacje mogą być wykonywane na danych oraz zwracają odpowiedzi w zunifikowanym formacie (`status`, `message`, `data`), zgodnie z wcześniej omówionymi zasadami.

Widoki łączą modele bazy danych z serializatorami i obsługują żądania HTTP, umożliwiając wykonywanie operacji CRUD (**C**reate, **R**ead, **U**ppdate, **D**elede).

Rodzaje widoków w Django REST Framework W Django REST Framework istnieje kilka sposobów implementacji widoków. W projekcie backendu dominują **widoki oparte na klasach (CBV – Class-Based Views)**, które zapewniają większą elastyczność, reużywalność kodu oraz lepszą organizację logiki API.

Przykład prostego widoku API dla kursów Poniższy przykład implementuje `APIView` do obsługi kursów (`Course`):

```
from rest_framework.response import Response
from rest_framework.views import APIView
from .models import Course
from .serializers import CourseSerializer

class CourseListView(APIView):
    def get(self, request):
        courses = Course.objects.all()
        serializer = CourseSerializer(courses, many=True)
        return Response({
            "status": "success",
            "message": "Courses retrieved successfully.",
            "data": serializer.data
        })
```

Ten widok:

- Pobiera wszystkie kursy z bazy danych.
- Serializuje je do formatu JSON.
- Zwraca odpowiedź zgodną z ujednoliconym formatem API.

4.9.5. urls.py

Plik `urls.py` odpowiada za routowanie, czyli mapowanie adresów URL na odpowiednie widoki. Jest kluczowym elementem nawigacji w aplikacji, ponieważ to do tych endpointów frontend wysyła żądania o dane.

Django obsługuje routing za pomocą listy `urlpatterns`, w której definiuje się trasy dla poszczególnych widoków API.

Przykład konfiguracji URL dla widoków obsługujących kursy:

```
from django.urls import path
from .views import CourseListCreateView, CourseDetailView

urlpatterns = [
    path("courses/", CourseListCreateView.as_view(),
        name="course-list"),
    path("courses/<int:pk>/", CourseDetailView.as_view(),
        name="course-detail"),
]
```

W powyższym przykładzie:

- `"courses/"` – obsługuje listowanie kursów i tworzenie nowych kursów.
- `"courses/<int:pk>/"` – obsługuje operacje na pojedynczym kursie (odczyt, aktualizacja, usunięcie), gdzie `<int:pk>` oznacza identyfikator kursu przekazywany jako parametr w URL.

4.9.6. settings.py

Plik `settings.py` to główne centrum konfiguracji projektu Django. Zawiera kluczowe ustawienia aplikacji, które wpływają na jej działanie, takie jak:

- **Konfiguracja bazy danych** – określa, z jaką bazą danych Django ma się łączyć.
- **Zainstalowane aplikacje** – lista aplikacji Django oraz aplikacji własnych (`INSTALLED_APPS`).
- **Ustawienia międzynarodowe** – konfiguracja języka, strefy czasowej itp.
- **Ustawienia bezpieczeństwa** – m.in. klucz `SECRET_KEY`, mechanizmy autoryzacji i zabezpieczenia przed atakami (np. CSRF, CORS).

- **Konfiguracja uwierzytelniania OAuth** – zawiera wszystkie zmienne potrzebne do przeprowadzenia procedury OAuth.
- **Ustawienia plików statycznych** – konfiguracja katalogów dla plików statycznych (STATIC_URL, STATICFILES_DIRS).
- **Dodatkowe ustawienia** – inne konfiguracje, takie jak obsługa e-maili, cache, czy logowanie błędów.

5. Wdrożenie aplikacji

Ostatecznym etapem rozwoju oprogramowania było jego wdrożenie. Przejście ze środowiska deweloperskiego do środowiska produkcyjnego stanowi kluczowy moment, ponieważ umożliwia użytkownikom końcowym pełne korzystanie z aplikacji. W celu wdrożenia zdecydowano się na wykorzystanie platformy Render.com [32], która oferuje darmowy hosting zarówno dla statycznych stron (frontendu), jak i aplikacji webowych (backendu Django). Co więcej, Render umożliwia integrację z bazą danych PostgreSQL [5].

5.1. Wybór bazy danych: PostgreSQL zamiast SQLite

W środowisku deweloperskim wykorzystywana była baza danych `sqlite3` [13]. Jest to domyślna baza danych Django, która sprawdza się świetnie w testowaniu aplikacji oraz przyspiesza pracę deweloperską, ponieważ nie wymaga skomplikowanej konfiguracji. Po wywołaniu poleceń:

```
python manage.py makemigrations
python manage.py migrate
```

Django automatycznie tworzy plik bazy danych `db.sqlite3`, eliminując konieczność konfiguracji zewnętrznego serwera bazy danych. Dzięki temu deweloper może szybko uruchomić i testować aplikację bez dodatkowych działań.

Jednak w środowisku produkcyjnym wymagania dotyczące bazy danych są znacznie bardziej restrykcyjne. **SQLite ma istotne ograniczenia, które czynią go niewystarczającym do aplikacji o większym ruchu:**

- **Problemy z równoczesnym dostępem** – SQLite blokuje całą bazę danych na czas operacji zapisu. Oznacza to, że jeśli wielu użytkowników korzysta z aplikacji jednocześnie, mogą występować problemy z wydajnością i poprawnością zapisu.
- **Brak zaawansowanego systemu zarządzania użytkownikami** – SQLite nie posiada wbudowanego mechanizmu zarządzania użytkownikami i kontrolowania dostępu do bazy danych, co czyni go mniej bezpiecznym.
- **Brak wsparcia dla bardziej zaawansowanych operacji SQL** – PostgreSQL oferuje bardziej rozbudowane możliwości w zakresie transakcji, indeksowania i optymalizacji zapytań, co jest kluczowe w aplikacjach produkcyjnych.

Z tych powodów w środowisku produkcyjnym zdecydowano się na wykorzystanie bazy danych **PostgreSQL**, która jest popularnym i stabilnym rozwiązaniem wykorzystywanym w dużych systemach produkcyjnych. PostgreSQL oferuje wysoką wydajność, obsługę zaawansowanych zapytań SQL oraz rozbudowane mechanizmy kontroli dostępu.

5.2. Zmienne środowiskowe – bezpieczeństwo i elastyczność

Niektóre informacje, takie jak klucze API, dane dostępowe do bazy danych czy ustawienia bezpieczeństwa, nie powinny być przechowywane bezpośrednio w kodzie źródłowym aplikacji. **Zmienne środowiskowe** pozwalają na bezpieczne przechowywanie tych wartości oraz umożliwiają łatwe przełączanie się między środowiskami deweloperskim a produkcyjnym.

Dlaczego zmienne środowiskowe?

- Zapewniają **bezpieczeństwo** – hasła, klucze API i poufne informacje nie są zapisane w repozytorium kodu, co zapobiega ich przypadkowemu ujawnieniu.
- Umożliwiają **łatwą konfigurację różnych środowisk** – dzięki nim można dynamicznie zmieniać ustawienia aplikacji w zależności od środowiska (lokalnego, testowego, produkcyjnego) bez konieczności edytowania kodu źródłowego.
- Pozwalają na **centralne zarządzanie konfiguracją** – zmiana wartości zmiennej w jednym miejscu wpływa na całą aplikację.

5.2.1. Struktura plików środowiskowych w aplikacji

W przypadku naszej aplikacji można wyróżnić pięć plików środowiskowych, które dzielą się na te używane lokalnie oraz te przechowywane w serwisie Render.com.

Render.com umożliwia dodanie zmiennych środowiskowych dla środowiska produkcyjnego podczas konfiguracji projektu. Dane te są szyfrowane i przechowywane w bezpieczny sposób. Zmienne środowiskowe w Render mają tę samą strukturę co plik konfiguracyjny backendu dla środowiska produkcyjnego, który można zobaczyć poniżej. Natomiast pozostałe cztery pliki są przechowywane lokalnie (ze względu na ich niedodawanie do repozytorium), dlatego ich konfiguracja w Render.com musi odbywać się ręcznie.

Aby umożliwić łatwe zarządzanie konfiguracją, stworzono cztery pliki konfiguracyjne – po dwa dla frontendu oraz backendu:

- **Frontend** – `.env.development`, `.env.production`

- **Backend** – `.env.development`, `.env.production`

Ich struktura zostanie omówiona na przykładzie plików `.env.development` zarówno dla frontendu, jak i backendu.

5.2.2. Plik środowiskowy dla frontendu

Plik `.env.development` zawiera jedynie adresy backendu wykorzystywane do komunikacji frontendu z backendem:

```
VITE_API_BASE_URL="http://localhost:8000"
VITE_API_ENDPOINT="/api"
```

Dzięki temu, zamiast twardego kodowania adresów API w kodzie aplikacji frontendowej, możliwe jest łatwe przełączanie środowisk poprzez zmianę wartości zmiennych.

5.2.3. Plik środowiskowy dla backendu

Plik `.env.development` backendu zawiera więcej parametrów, w tym dane konfiguracyjne Django, bazę danych oraz ustawienia CORS:

```
DEBUG=True
SECRET_KEY="secret-key"
ALLOWED_HOSTS=localhost,127.0.0.1
DATABASE_URL=sqlite:///db.sqlite3
CORS_ALLOWED_ORIGINS=http://localhost:8000,http://localhost:5173,
    https://usosapi.polsl.pl
CSRF_TRUSTED_ORIGINS=http://localhost:8000,http://localhost:5173,
    https://usosapi.polsl.pl

USOS_BASE_URL=https://usosapi.polsl.pl
USOS_CONSUMER_KEY=consumer-key
USOS_CONSUMER_SECRET=consumer-secrets
USOS_REQUEST_TOKEN_URL=
    https://usosapi.polsl.pl/services/oauth/request_token
USOS_AUTHORIZE_URL=
    https://usosapi.polsl.pl/services/oauth/authorize
USOS_ACCESS_TOKEN_URL=
    https://usosapi.polsl.pl/services/oauth/access_token
USOS_CALLBACK_URL=http://localhost:8000/api/oauth/callback/
FRONTEND_URL=http://localhost:5173
```

5.2.4. Omówienie wybranych zmiennych

Niektóre z powyższych zmiennych pełnią kluczowe funkcje w konfiguracji aplikacji:

- **SECRET_KEY** – klucz zabezpieczający aplikację Django.
- **ALLOWED_HOSTS** – lista dozwolonych hostów, które mogą łączyć się z aplikacją (więcej na ten temat w sekcji **SOP i CORS**).
- **DATABASE_URL** – adres bazy danych. W środowisku deweloperskim aplikacja korzysta z SQLite (`sqlite:///db.sqlite3`), natomiast w produkcji z PostgreSQL (`postgres://user:password@host:port/dbname`).
- **CORS_ALLOWED_ORIGINS** – lista adresów URL, z których dozwolone są zapytania do backendu.
- **USOS_BASE_URL** oraz powiązane z nim adresy – kluczowe adresy wykorzystywane w autoryzacji OAuth 1.0a w systemie USOS API.

5.2.5. Pobieranie zmiennych środowiskowych w Django

Aby aplikacja mogła korzystać z wartości zapisanych w plikach `.env`, Django wykorzystuje moduł `os`, który umożliwia dynamiczne odczytywanie zmiennych środowiskowych.

Przykładowa implementacja:

```
import os
```

```
SECRET_KEY = os.getenv('SECRET_KEY')
```

```
ALLOWED_HOSTS = os.getenv('ALLOWED_HOSTS', '').split(',')
```

`os.getenv('SECRET_KEY')` pobiera wartość zmiennej środowiskowej `SECRET_KEY`, a `os.getenv('ALLOWED_HOSTS', '').split(',')` pozwala na ustawienie wielu dozwolonych hostów poprzez zapis w postaci listy oddzielonej przecinkami.

5.2.6. Korzyści wynikające z zastosowania plików `.env`

Pliki `.env` pełnią kluczową rolę w zarządzaniu konfiguracją aplikacji:

- Zapewniają **bezpieczeństwo**, eliminując ryzyko ujawnienia wrażliwych danych w kodzie źródłowym.
- Umożliwiają **łatwą zmianę środowiska** – wystarczy odpowiednio skonfigurować zmienne, aby aplikacja działała poprawnie zarówno lokalnie, jak i w środowisku produkcyjnym.
- Pozwalają na **separację konfiguracji** – oddzielne pliki dla frontendu i backendu zapewniają większą przejrzystość i kontrolę nad ustawieniami aplikacji.

Dzięki wykorzystaniu zmiennych środowiskowych aplikacja staje się bardziej elastyczna, bezpieczna i łatwiejsza w zarządzaniu, zarówno podczas procesu deweloperskiego, jak i w trakcie wdrożenia produkcyjnego.

5.3. Budowanie frontendu i integracja z backendem

Ze względu na to, że aplikacja nie jest bardzo rozbudowana, podjęto decyzję o wdrożeniu frontendowej części aplikacji jako statycznych plików obsługiwanych bezpośrednio przez Django. Takie podejście pozwoliło na:

- umieszczenie zarówno backendu, jak i frontendowych zasobów w jednym miejscu pod tym samym adresem URL,
- uniknięcie problemów związanych z ograniczeniami CORS oraz przekazywaniem plików cookie między różnymi domenami.

Aby móc serwować pliki frontendowe poprzez Django, konieczne było ich wygenerowanie i umieszczenie w katalogu backendu. Proces ten składał się z następujących kroków:

1. Wykonanie komendy `npm run build`, która wygeneruje statyczne pliki `index.html`, arkusze stylów CSS, skrypty JavaScript oraz inne zasoby, takie jak favicon czy obrazy.
2. Przeniesienie wygenerowanych plików do nowo utworzonego katalogu `static` w projekcie Django, z wyjątkiem `index.html`, który musiał znajdować się w katalogu `templates`.

3. Modyfikacja pliku `index.html` w celu poprawnego odnalezienia plików statycznych przez Django, poprzez dodanie:

```
{% load static %}
```

oraz zamianę ścieżek do plików statycznych na:

```
"{% static "ścieżka/do/pliku" %}"
```

5.4. Konfiguracja ciągłego wdrożenia (Continuous Deployment)

Render.com umożliwia automatyczne wdrażanie aplikacji na podstawie zmian w systemie kontroli wersji. W tym celu wykonano następujące kroki:

- Gotowy kod aplikacji został dodany do repozytorium Git w gałęzi `main`.
- W Render.com podano link do repozytorium, przeprowadzono autoryzację oraz nadano aplikacji dostęp do kodu źródłowego.
- Ustawiono folder główny aplikacji na `/backend`, ponieważ projekt jest monorepozytorium (zawiera zarówno frontend, jak i backend w osobnych folderach).
- Skonfigurowano plik pre-wdrożeniowy zawierający polecenia wykonywane przed uruchomieniem aplikacji.

5.5. Plik pre-wdrożeniowy `build.sh`

Render.com wymaga określenia poleceń, które powinny zostać wykonane przed wdrożeniem aplikacji. W tym celu utworzono skrypt `build.sh`, który zawiera:

```
# Exit on error
set -o errexit

pip install -r requirements.txt
python manage.py collectstatic --noinput
python manage.py migrate
```

Działanie poszczególnych komend:

- **pip install -r requirements.txt** – instaluje wszystkie wymagane pakiety Django zgodnie z wersjami zapisanymi w pliku `requirements.txt`. Plik ten można wygenerować za pomocą:

```
pip freeze > requirements.txt
```

Pozwala to na zachowanie spójności wersji pakietów między środowiskiem deweloperskim a produkcyjnym.

- **python manage.py collectstatic --noinput** – zbiera wszystkie pliki statyczne aplikacji, w tym zarówno pliki frontendowe, jak i backendowe, i umieszcza je w katalogu wskazanym w ustawieniach Django (`STATIC_ROOT`).
- **python manage.py migrate** – wykonuje migracje bazy danych na podstawie istniejących plików migracyjnych, tworząc odpowiednie tabele w bazie danych.

5.6. Uruchamianie aplikacji na Render.com

Aby Render.com mógł uruchomić aplikację, konieczne było podanie komendy startowej. W przypadku Django wykorzystano serwer Gunicorn w połączeniu z Uvicorn:

```
python -m gunicorn student_grades.asgi:application
-k uvicorn.workers.UvicornWorker
```

Gunicorn [33] to serwer HTTP, który służy do uruchamiania aplikacji Django w środowisku produkcyjnym. Odpowiada za obsługę żądań użytkowników i zarządzanie procesami aplikacji.

Uvicorn to szybki i lekki serwer przeznaczony do pracy z aplikacjami asynchronicznymi, które wykorzystują nowoczesne rozwiązania, takie jak `asyncio`. W naszej aplikacji Gunicorn używa Uvicorn jako "silnika", co pozwala na bardziej wydajną obsługę wielu zapytań jednocześnie.

5.6.1. Plik ASGI w Django

Plik `asgi.py` odpowiada za uruchomienie aplikacji zgodnie z ustawieniami zapisanymi w `settings.py`:

```
import os
from django.core.asgi import get_asgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
    "student_grades.settings")
application = get_asgi_application()
```

Dzięki zastosowaniu ASGI, aplikacja może obsługiwać asynchroniczne żądania, co jest istotne w nowoczesnych systemach webowych wymagających lepszej skalowalności i wydajności.

5.6.2. Testowanie wdrożenia na Render.com

Po zakończeniu procesu budowania Render uruchamia aplikację oraz testuje jej działanie, wysyłając próbne zapytanie HTTP do wdrożonej instancji. Jeśli serwer zwróci kod odpowiedzi 200 OK, oznacza to, że aplikacja działa poprawnie i staje się publicznie dostępna pod przypisanym adresem URL.

5.7. SOP i CORS

Jak zostało już wspomniane we wcześniejszym podpunkcie, należało również spełnić wymagania bezpieczeństwa, które obecnie są egzekwowane przez przeglądarki. Jednym z najważniejszych mechanizmów jest **SOP** [34] (*Same-Origin Policy*) [35].

Origin można rozumieć jako „pochodzenie” strony, czyli kombinację protokołu (np. `http` lub `https`), hosta (np. `www.polsl.pl`) oraz portu (np. `:443`). Przeglądarki stosują politykę SOP, która zabrania aplikacjom pochodzącym z różnych źródeł (ang. *origin*) wzajemnego odczytywania swoich zasobów. Oznacza to, że jeśli frontend działa na `https://frontend.example.com`, a backend na `https://backend.example.com`, przeglądarka domyślnie zablokuje możliwość wymiany danych między tymi serwisami.

5.7.1. CORS – kontrolowane udostępnianie zasobów

Polityka **CORS** (*Cross-Origin Resource Sharing*) pozwala na kontrolowane udostępnianie zasobów między różnymi originami. Mechanizm ten został wprowadzony, aby umożliwić przeglądarkom bezpieczną wymianę danych między aplikacjami frontendowymi i backendowymi, które znajdują się pod różnymi domenami. Działa on poprzez dodanie specjalnych nagłówków HTTP informujących przeglądarkę, które origini mają dostęp do zasobów danego serwera.

W celu zapewnienia poprawnej komunikacji aplikacji z API USOS, ustawienia CORS zostały zredukowane do minimum, aby aplikacja mogła działać pod jednym adresem URL oraz komunikować się bezpośrednio z API systemu USOS.

5.7.2. Alternatywne rozwiązanie i problem XSS

W tym miejscu pojawia się pytanie – dlaczego nie wdrożono standardowego rozwiązania, w którym frontend jest hostowany oddzielnie, a backend pośredniczy w komunikacji z USOS API? Wydawałoby się, że takie podejście również byłoby zgodne z CORS, ponieważ frontend łączyłby się tylko z backendem, a backend komunikowałby się z USOS API. Jednak to rozwiązanie niesie ze sobą zagrożenie atakami **XSS** (*Cross-Site Scripting*) [36].

XSS to rodzaj ataku, w którym złośliwy kod JavaScript jest wstrzykiwany do aplikacji webowej. Może to prowadzić do przejęcia sesji użytkownika, wykradania danych lub manipulacji interfejsem aplikacji. W przypadku, gdy frontend i backend są hostowane oddzielnie, a frontend wykonuje dynamiczne zapytania do backendu, istnieje ryzyko, że niezabezpieczone żądania mogą zostać przechwycone i zmanipulowane przez atakującego.

Dodatkowo w tym scenariuszu problemem mogą być **third-party cookies** [37], czyli ciasteczka należące do innej domeny niż ta, na której zostały pierwotnie utworzone. Przeglądarki coraz częściej stosują politykę blokowania takich ciasteczek, co mogłoby utrudnić prawidłowe działanie mechanizmu sesji. Przykładowo, jeśli identyfikator sesji użytkownika (zapisany w ciasteczku) jest przypisywany podczas logowania przez backend i następnie przesyłany do frontendowej domeny, może się okazać, że przeglądarka zablokuje jego odczyt ze względu na restrykcje dotyczące ciasteczek stron trzecich. W rezultacie rozpoznanie zalogowanego użytkownika mogłoby stać się niemożliwe.

Wdrożenie frontendu jako statycznych plików serwowanych przez backend Django znacząco zmniejsza ryzyko XSS, ponieważ:

- wszystkie żądania pochodzą z tego samego originu, eliminując potrzebę stosowania mechanizmu CORS,
- zabezpieczenia Django ograniczają możliwość wstrzyknięcia złośliwego kodu,
- nie dochodzi do sytuacji, w której frontend wysyła nieautoryzowane żądania do backendu z różnych źródeł.

5.7.3. Praktyki zapewniające bezpieczeństwo

Aby zapewnić bezpieczeństwo użytkownikom aplikacji, wdrożono szereg mechanizmów spełniających wymagania przeglądarek i standardów bezpieczeństwa:

- **Same-Origin Policy (SOP)** – ogranicza dostęp do zasobów między różnymi originami,
- **Cross-Origin Resource Sharing (CORS)** – pozwala na bezpieczne udostępnianie zasobów między aplikacjami,
- **Ochrona przed XSS** – zmniejszenie ryzyka ataków poprzez serwowanie frontendu jako statycznych plików przez Django.

Dzięki tym rozwiązaniom aplikacja zapewnia wysoki poziom bezpieczeństwa, jednocześnie spełniając wymagania przeglądarek dotyczące polityki dostępu do zasobów.

6. Narzędzia wykorzystane w procesie rozwoju aplikacji

6.1. Git

System kontroli wersji oparto na systemie Git, a repozytorium zostało utworzone na portalu GitHub. Dostęp do niego mieli wyłącznie twórcy aplikacji. W ramach projektu utworzono cztery gałęzie: `frontend`, `backend`, `development` i `main`.

W początkowej fazie rozwoju, gdy `frontend` i `backend` działały niezależnie, każdy programista dodawał kod do swojej dedykowanej gałęzi. Po podjęciu decyzji o integracji kod źródłowy z gałęzi `frontend` i `backend` został scalony w gałęzi `development`, która stała się główną gałęzią w tej fazie rozwoju aplikacji.

Gdy zapadła decyzja o wdrożeniu aplikacji, kod z gałęzi `development` został scalony z `main`, która stała się główną gałęzią wdrożeniową – to z niej kod był pobierany przez serwis `render.com` do wdrożenia.

Na moment pisania pracy repozytorium zawiera **77 commitów**.

6.2. Kanban w Taiga.io

Podczas pracy nad projektem wykorzystywano narzędzie Taiga.io, które służy do zarządzania z wykorzystaniem metodyk agile (zwinnych) w ramach Scrum i Kanban. Taiga jest platformą open-source, umożliwiającą organizację zadań, śledzenie postępów oraz efektywną współpracę zespołu. Zdobyła nagrodę „The Best Agile Tool” na Agile Awards 2015 oraz została wymieniona jako jedno z 7 najlepszych narzędzi do zarządzania projektami na 2020 rok przez OpenSource.com [22].

Kluczowym elementem używanym w ramach tego narzędzia była **tablica Kanban**, pozwalająca na wizualizację procesu pracy. Zadania były podzielone na kolumny odpowiadające różnym etapom realizacji projektu: „Nowe” (New), „Gotowe” (Ready), „W trakcie” (In Progress), „Gotowe do testowania” (Ready for test) oraz „Zakończone” (Done). Dzięki temu można było na bieżąco monitorować postęp prac, komunikować się dodając zadania i komentarze oraz szybko reagować na ewentualne przeszkody.

Taiga.io umożliwia również priorytetyzację zadań oraz przypisywanie ich jednemu z dwóch członków zespołu. Wykorzystanie tego narzędzia pozwoliło na bardziej uporządkowaną pracę i lepszą koordynację działań.

6.3. Web Inspector

Web Inspector to wbudowane w przeglądarki narzędzie deweloperskie, umożliwiające analizę działania aplikacji webowych. Posiada szereg funkcji wspomagających debugowanie kodu i optymalizację działania strony.

- **Inspekcja elementów HTML i JavaScript** – pozwala na podgląd oraz edycję struktury HTML, arkuszy stylów CSS oraz wykonywanego kodu JavaScript.
- **Konsola JavaScript** – umożliwia monitorowanie błędów, testowanie fragmentów kodu oraz wyświetlanie komunikatów zaprogramowanych przez programistę.
- **Zakładka Network** – pozwala analizować ruch sieciowy oraz ładowane zasoby (np. zapytania HTTP, odpowiedzi API, czas ładowania).
- **Zakładka Storage** – umożliwia przeglądanie zapisanych ciasteczek (*cookies*), `localStorage` oraz `sessionStorage`.

Funkcje te były niezwykle pomocne podczas tworzenia aplikacji, umożliwiając szczegółową analizę działania kodu i diagnozowanie problemów.

Testy w różnych przeglądarkach Aplikacja została przetestowana pod kątem poprawnego działania w najpopularniejszych przeglądarkach internetowych:

- **Google Chrome** (Windows, macOS)
- **Safari** (macOS)
- **Microsoft Edge** (Windows)

Każda z tych przeglądarek posiada własne ustawienia i mechanizmy bezpieczeństwa, które mogą wpływać na sposób działania aplikacji. Przykładowo, Safari stosuje bardziej restrykcyjne polityki dotyczące ciasteczek oraz przechowywania danych w `localStorage` niż Chrome, co powodowało różnice w obsłudze sesji użytkownika.

6.4. Postman

Postman to niezastąpione narzędzie do testowania API, umożliwiające wysyłanie żądań HTTP do serwera oraz analizę otrzymanych odpowiedzi. Jest szczególnie przydatne przy testowaniu endpointów, pozwalając na szybkie sprawdzenie poprawności działania metod CRUD (**C**reate, **R**ead, **U**ppdate, **D**elele).

Postman oferuje:

- **Przejrzysty interfejs użytkownika** – ułatwia tworzenie i organizowanie zapytań.
- **Obsługę różnych metod HTTP** – GET, POST, PUT, DELETE i innych.
- **Symulację żądań API** – pozwala na testowanie różnych scenariuszy aplikacji.
- **Obsługę autoryzacji** – wspiera metody uwierzytelniania, takie jak OAuth
- **Możliwość tworzenia kolekcji zapytań** – ułatwia grupowanie i ponowne wykorzystanie testów API.

Postman znacząco ułatwił proces testowania API w tym projekcie, umożliwiając szybkie wykrywanie i diagnozowanie błędów.

6.5. Debugger VSCode

Visual Studio Code (*VSCode*) posiada wbudowane narzędzie do debugowania, które umożliwia analizę kodu krok po kroku. Pozwala ono na śledzenie wartości zmiennych, ich zawartości oraz zmian w czasie rzeczywistym podczas działania debuggera.

Najważniejsze funkcje debugowania w VSCode:

- **Przechwytywanie błędów** – możliwość zatrzymywania wykonywania kodu w miejscach występowania błędów.
- **Śledzenie zmiennych** – podgląd wartości zmiennych i ich modyfikacja „na gorąco” w trakcie działania debuggera.
- **Obserwowanie przepływu programu** – analiza kolejności wykonywania instrukcji i wpływu poszczególnych operacji na kod.
- **Ustawianie punktów przerwań (breakpoints)** – zatrzymanie wykonania kodu w określonym miejscu, co pozwala na jego dokładną analizę.

Dzięki tym funkcjom debugowanie w VSCode znacząco ułatwia analizę kodu, wykrywanie błędów oraz lepsze zrozumienie zależności między różnymi elementami programu.

7. Podsumowanie

Projekt zrealizował kluczowe założenia, dostarczając w pełni funkcjonalne narzędzie do zarządzania kursami w środowisku akademickim. Aplikacja umożliwia tworzenie kursów, zarządzanie ich wymaganiami, rejestrację studentów, a także przyznawanie ocen. Dzięki wdrożeniu systemu na serwerze jest on ogólnodostępny i nie wymaga dodatkowej konfiguracji po stronie użytkownika, co znacząco zwiększa jego użyteczność.

Jednym z kluczowych elementów aplikacji jest moduł zarządzania kursami, który pozwala nauczycielom definiować wymagania zaliczeniowe, a studentom monitorować postępy. Proces logowania użytkowników został uproszczony dzięki integracji z systemem USOS, co eliminuje konieczność ręcznego tworzenia kont. Moduł oceniania zapewnia przejrzysty system przyznawania ocen, jednak nie przewiduje możliwości zmian w wymaganiach, co jest zgodne z ogólnymi zasadami akademickimi.

Pomimo pełnej funkcjonalności aplikacji istnieje możliwość jej dalszej rozbudowy i optymalizacji. Pierwszym aspektem, który można ulepszyć, jest infrastruktura serwera, np. poprzez wdrożenie szybszego hostingu oraz rejestrację dedykowanej domeny, co poprawiłoby wydajność i profesjonalny wygląd systemu. Warto również rozważyć dodanie podstawowych testów, które mogłyby zwiększyć niezawodność aplikacji i ułatwić jej przyszły rozwój, choć obecnie system działa stabilnie i nie wykazuje problemów wymagających szczegółowej walidacji.

Pod kątem samego oprogramowania można wprowadzić dodatkowe funkcje, które zwiększyłyby jego elastyczność:

- **Możliwość wyrejestrowania studenta z kursu**, co pozwoliłoby na lepsze zarządzanie listą uczestników.
- **Zabezpieczenie kursów przed nieautoryzowanym dostępem**, np. poprzez system haseł dających dostęp do kursów lub akceptację przez prowadzącego.
- **System restartu kursów między semestrami**, umożliwiający usunięcie wszystkich zapisanych studentów i edycję elementów wymaganych do zaliczenia.
- **Funkcja ukrywania kursów**, co pozwoliłoby nauczycielom na przygotowanie ich z wyprzedzeniem bez konieczności udostępniania studentom przed rozpoczęciem semestru.

Współpraca z USOS API dostarczyła wielu informacji na temat funkcjonowania tego akademickiego systemu. Poznanie jego złożoności skłoniło do refleksji nad kwestią integracji z innymi narzędziami wykorzystywanymi na uczelniach. Przykładem jest wykorzystanie popularnego środowiska edukacyjnego Moodle przez Politechnikę Śląską (<https://platforma.polsl.pl/>), które obecnie nie jest bezpośrednio powiązane z USOS. Potencjalna integracja mogłaby uprościć procesy administracyjne, np. automatyczną rejestrację studenta na kurs w USOS po zapisaniu się na niego w systemie e-learningowym.

Znaleziono kilka podobnych przykładów zastosowań, które mogłyby usprawnić pracę zarówno studentów, jak i nauczycieli, oszczędzając ich czas. Obecnie systemy te funkcjonują jako odrębne byty, co wymaga wykonywania wielu czynności administracyjnych manualnie.

Aplikacja opracowana w ramach tej pracy jest prostą alternatywą dla złożonych platform edukacyjnych, wykorzystującą jedynie niewielką część możliwości oferowanych przez USOS API. Może jednak stanowić inspirację do dalszego wykorzystania jego potencjału i budowy bardziej zintegrowanych narzędzi, które usprawniłyby elektroniczną obsługę studiów, czyniąc ją szybszą i wygodniejszą dla wszystkich użytkowników.

Podsumowując, aplikacja spełnia swoje główne założenia i może być wykorzystywana w środowisku akademickim jako narzędzie wspomagające proces dydaktyczny. Jednocześnie jej struktura została zaprojektowana tak, aby umożliwiać łatwą rozbudowę, co pozwala na dalszy rozwój i dostosowanie do zmieniających się potrzeb użytkowników.

Literatura

- [1] MDN Web Docs (Mozilla), *JavaScript – Dokumentacja MDN*, <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [2] Zespół React (Meta), *React – Oficjalna dokumentacja*, <https://react.dev/>
- [3] Django Software Foundation, *Dokumentacja Django*, Dostępne na: <https://docs.djangoproject.com/en/5.1/>
- [4] Fielding, Roy T., *Architectural- Styles and the Design of Networkbased Software Architectures*, University of California, Irvine, 2000. Dostępne na: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [5] Zespół PostgreSQL Global Development Group, *PostgreSQL – Oficjalna dokumentacja*, <https://www.postgresql.org/>
- [6] Zespół Git, *Git – Oficjalna dokumentacja*, <https://git-scm.com/>
- [7] GitHub, Inc., *GitHub – Platforma do hostingu kodu*, <https://github.com/>
- [8] Moodle Pty Ltd, *Moodle – Platforma e-learningowa*, <https://moodle.com/about/>
- [9] SchoolStatus, LLC, *SchoolStatus – Platforma dla edukacji*, <https://www.schoolstatus.com/about>
- [10] npm, Inc., *npm – Node Package Manager*, <https://www.npmjs.com/>
- [11] Python Software Foundation, *Python Documentation*, Dostępne na: <https://docs.python.org/3/>
- [12] Python Software Foundation, *pip – The Python Package Installer*, <https://pip.pypa.io/>
- [13] SQLite, *SQLite Documentation*, Dostępne na: <https://www.sqlite.org/docs.html>
- [14] Zespół React (Meta), *React DOM – Oficjalna dokumentacja*, <https://react.dev/reference/react-dom>

- [15] Zespół React (Meta), *React Hooks – Oficjalna dokumentacja*, <https://react.dev/reference/react/hooks>
- [16] Zespół Vite, *Vite – Oficjalna dokumentacja*, <https://vitejs.dev/>
- [17] Color Hunt, *Color Hunt – Curated Color Palettes*, <https://colorhunt.co/>
- [18] Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F., *Computer Graphics: Principles and Practice* (3rd ed.), Addison-Wesley, 2013.
- [19] Stefanov, S., *JavaScript. Programowanie obiektowe*, Helion, 2010.
- [20] Zespół Vite, *Vite – Konfiguracja Proxy*, <https://vitejs.dev/config/#server-proxy>
- [21] Sochacki, T., *JavaScript. Techniki zaawansowane*, Helion, 2018.
- [22] Taiga.io, *Taiga – Agile Project Management Platform*, <https://www.taiga.io/>
- [23] Python Software Foundation, *venv – Creation of Virtual Environments*, Dostępne na: <https://docs.python.org/3/library/venv.html>
- [24] Encode, *Dokumentacja Django REST Framework*, Dostępne na: <https://www.django-rest-framework.org/>
- [25] Mozilla Developer Network (MDN), *Single Page Application (SPA) – Przewodnik*, Dostępne na: <https://developer.mozilla.org/en-US/docs/Glossary/SPA>
- [26] Elmasri, R., Navathe, S.B., *Fundamentals of Database Systems*, 7th ed., Pearson Education, 2016.
- [27] Django Software Foundation, *Django Database Models – Official Documentation*, Dostępne na: <https://docs.djangoproject.com/en/5.1/topics/db/models/>
- [28] Django Software Foundation, *Primary Key Fields in Django*, Dostępne na: https://docs.djangoproject.com/en/5.1/ref/models/fields/#django.db.models.Field.primary_key
- [29] Django Software Foundation, *Django Admin Site – Official Documentation*, Dostępne na: <https://docs.djangoproject.com/en/5.1/ref/contrib/admin/>
- [30] OAuth Core Workgroup, *OAuth 1.0 Core Specification*, Dostępne na: <https://oauth.net/core/1.0/>

- [31] Politechnika Śląska, *Dokumentacja USOS API*, Wersja 7.1.1.0-1, f733442d, dirty (2024-11-25). Dostępne na: <https://usosapi.polsl.pl/developers/api/>
- [32] Render, *Render.com Documentation*, Dostępne na: <https://render.com/docs>
- [33] Gunicorn Team, *Dokumentacja Gunicorn*, Dostępne na: <https://docs.gunicorn.org/en/stable/>
- [34] Mozilla Developer Network (MDN), *Same-Origin Policy (SOP)*, Dostępne na: https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy
- [35] Mozilla Developer Network (MDN), *Cross-Origin Resource Sharing (CORS)*, Dostępne na: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- [36] Open Web Application Security Project (OWASP), *Cross-Site Scripting (XSS)*, Dostępne na: <https://owasp.org/www-community/attacks/xss/>
- [37] Mozilla Developer Network (MDN), *HTTP Cookies*, Dostępne na: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>